

PEBBLING GAME AND ALTERNATIVE BASIS FOR HIGH PERFORMANCE MATRIX MULTIPLICATION*

ODED SCHWARTZ[†] AND NOA VAKNIN[†]

Abstract. Matrix multiplication is one of the most extensively used kernels in scientific computing. Although subcubic algorithms exist, most high performance implementations are based on the classical $\Theta(n^3)$ matrix multiplication. Designing an algorithm that obtains even modest improvements in performance over existing implementations, requires carefully addressing challenges such as reducing computation costs, communication costs, and memory footprint. We provide the first high performance general matrix-matrix multiplication that utilizes the alternative basis method on Strassen’s algorithm. We reduce the basis transformation overheads and decrease the memory footprint of the bilinear phase by using the pebbling game optimization scheme, consequentially improving both arithmetic and communication costs. Our algorithm outperforms DGEMM on feasible matrix dimensions starting at $n = 96$. It obtains an increasing speedup of up to nearly $\times 2$ speedup for larger matrix dimensions when running sequentially, and even larger speedups for certain matrix dimensions when running in parallel.

Key words. fast matrix multiplication, bilinear algorithm, pebbling game, alternative basis method

MSC codes. 15-04, 65F99, 65K05

DOI. 10.1137/22M1502719

1. Introduction. Matrix multiplication is a fundamental computation kernel in a large number of scientific computing applications. Consequentially, many efforts have been invested in improving its performance. Hardware vendors provide machine-specific highly optimized basic linear algebra subprograms (BLAS) implementations [34] such as Intel’s MKL [2], AMD’s AOCL [3] for CPUs, and Nvidia’s cuBLAS [4] for GPUs. Level 3 BLAS matrix multiplication is extensively used in high performance numerical linear algebra libraries such as LAPACK [8], and ScaLAPACK [22] amongst others. Hardware accelerators of matrix multiplication include Google’s TPU [52] and Habana Labs’ Gaudi [67], and are extensively used for high performance AI computations. Surprisingly, even though subcubic algorithms exist, almost all of these high performance matrix multiplication are based on classical $\Theta(n^3)$ algorithms. Fast matrix multiplication algorithms are normwise stable, as opposed to elementwise stability of the classical algorithm [12, 31, 43, 83]. For many applications, however, this suffices.

Obtaining an even moderate speedup in performance over existing matrix multiplication implementations is nontrivial. There are several challenges to consider, including reducing arithmetic computations, minimizing communication costs within the memory hierarchy, and between processors and improving memory footprint.

*Submitted to the journal’s Software and High-Performance Computing section June 16, 2022; accepted for publication (in revised form) July 26, 2023; published electronically November 15, 2023.
<https://doi.org/10.1137/22M1502719>

Funding: This work was supported by the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation programme grant 818252, and by computing time awarded on the Cyclone supercomputer of the High Performance Computing Facility of The Cyprus Institute under project 135. This research was supported by the Israel Science Foundation (grant 1354/23).

[†]The Rachel and Selim Benin School of Computer Science and Engineering, The Hebrew University of Jerusalem, Jerusalem, Israel (odedsc@cs.huji.ac.il, noa.vaknin1@mail.huji.ac.il).

We review previous work and our contribution, separately for implementation, arithmetic costs, memory footprint, and communication costs.

1.1. Previous implementations. Strassen’s $\Theta(n^{\log_2 7})$ algorithm [79] and Winograd’s improvement [72, 86] have been implemented (cf. [6, 10, 11, 27, 30, 33, 35, 40, 41, 46, 47, 48, 54, 57, 58, 62, 64, 65, 82]), as well as other fast matrix multiplication algorithms [19, 45, 53]. Kaporin [53] implemented a fast matrix multiplication algorithm based on Laderman, Pan, and Sha’s $\langle 48, 48, 48; 46464 \rangle$ -algorithm¹ [60], that obtained competitive performance with Strassen’s algorithm.

Gepner, Gamayunov, and Fraser [40] evaluated both Strassen’s algorithm and Intel’s MKL DGEMM. When utilizing all 16 available threads, their implementation of Strassen’s algorithm outperformed DGEMM. For matrix dimension $n = 32768$, they obtained an improvement of 33% compared to DGEMM.

Ballard and Benson [19] discovered new fast matrix multiplication algorithms, and implemented them using their automatic optimization tool. When multiplying square matrices, Strassen’s algorithm mostly obtained best performance, both sequentially and on 6 cores. For sufficiently large matrix dimensions, Strassen’s algorithm obtained an improvement of $\approx 20\%$ compared to DGEMM, running sequentially. When multiplying square matrices on 24 cores, Strassen’s algorithm outperformed DGEMM by $\approx 5\%$, and obtained competitive performance with the $\langle 4, 3, 3; 29 \rangle$ -algorithm, and the $\langle 4, 2, 4; 26 \rangle$ -algorithm.

Huang et al. [46] implemented several versions of Strassen’s algorithm using either one or two recursive steps before switching to a classical algorithm, and differing from each other in their use of memory. On a single core of the dual-socket Intel Xeon E5-2680 v2 processor, when multiplying sufficiently large square matrices, their implementations outperformed Intel’s MKL DGEMM. Their best performance reached ≈ 33 effective GFLOPS, compared to ≈ 27 by DGEMM. When running on 10 cores (a single socket) their improvements compared to DGEMM degraded.

Karppa and Kaski [54] implemented Boolean and binary matrix multiplication on GPUs, utilizing a new decomposition of Karstadt and Schwartz’s method [55] for Strassen’s algorithm [79].

Our contribution. We implement fast general matrix-matrix multiplication based on Strassen’s algorithm [79]. We utilize both the pebbling game [23, 49] for reducing memory footprint and the alternative basis method of [55], with additional improvements for reducing arithmetic and communication costs. To the best of our knowledge, this is the first high performance implementation utilizing the alternative basis method for general matrices, on CPUs. Benchmarked on Intel Xeon E5-2680, our algorithm outperforms Intel’s MKL DGEMM starting at matrix dimension $n = 96$, for all of the measured dimensions. We demonstrate a speedup of up to $\times 1.948$ compared to the runtime of DGEMM (improvement of 48.6%), for matrix dimension $n = 46080$ (see Figure 2(a)). Benchmarked on Intel Xeon Gold 6248, our algorithm outperforms Intel’s MKL DGEMM with significant speedups on small matrices, up 90.07% for $n = 448$ (see Figure 5(b)). To this end, we address the following challenges.

1.2. Arithmetic costs. Since the first subcubic matrix multiplication algorithm by Strassen [79], many algorithms further reduced the asymptotic complexity of matrix multiplication (cf. [7, 19, 28, 29, 38, 44, 51, 60, 61, 63, 69, 70, 71, 74, 75, 78, 80, 85, 86]). Unfortunately, many have large base case sizes and huge hidden constants (e.g., [28, 29, 63, 78, 80, 85]) thus rendering them impractical. However, other fast

¹See Notation 2.5.

TABLE 1
Arithmetic complexities.

Algorithm	Arithmetic complexity	Minimal matrix dimension
Strassen–Winograd [86]	$6n^{\log_2 7} - 5n^2$	—
Karstadt and Schwartz [55]	$5n^{\log_2 7} - 4n^2 + 3n^2 \log_2 n$	$n_0 = 32$
Here	$5n^{\log_2 7} - 4n^2 + 1.5n^2 \log_2 n$	$n_0 = 16$

The minimal matrix dimension refers to the dimension from which it is arithmetically worth using the specified algorithm, compared to using [86].

matrix multiplication algorithms (e.g., [17, 18, 19, 26, 44, 51, 55, 60, 61, 69, 71, 74, 75, 86]), maintain manageable base case sizes and hidden constants.

Cenk and Hasan [26] reduced the leading coefficient of Strassen–Winograd’s algorithm by using a nonuniform divide-and-conquer pattern and reusing sums. However, their method increases memory footprint and communication costs.

Karstadt and Schwartz [55] introduced an alternative basis method² that enables speeding up bilinear algorithms by a constant factor, in exchange for an $O(n^2 \log n)$ cost. They applied this method to many fast matrix multiplication algorithms, including Strassen’s $\langle 2, 2, 2; 7 \rangle$ -algorithm for which they obtained an optimal leading coefficient 5, as well as a speedup in communication costs.

Karppa and Kaski [54] utilized the alternative basis method of [55] in their high performance implementation of Boolean and binary matrix multiplication. They suggested a new decomposition of Strassen’s $\langle 2, 2, 2; 7 \rangle$ -algorithm over a binary field, that uses 2 linear operations³ in each basis transformation, compared to 4 of [55], and computed these basis transformations in place, thus reducing memory footprint.

Beniamini and Schwartz [18] generalized the alternative basis method of [55], by using transformations into intermediate dimension linear spaces, rather than automorphisms over the same linear space, as in [55]. This allowed them to further reduce the leading coefficients, for some algorithms even down to 2 (same as for the classical algorithm), but in exchange for larger low order terms.

Our contribution. We present a new decomposition of the $\langle 2, 2, 2; 7 \rangle$ -algorithm (see (3.1)), that generalizes Karppa and Kaski’s [54] alternative basis decomposition from a binary field to a general ring. This decomposition uses fewer linear operations in each basis transformation compared to [55], while maintaining the same number of linear operations in the bilinear phase. As a result, the basis transformation coefficients of both the arithmetic and communication costs are reduced, without incurring additional costs. Hence, the alternative basis method becomes worth applying already at smaller matrix dimensions (see Table 1).

1.3. Memory footprint. Strassen’s algorithm requires memory for storing intermediate computations, which increases the communication costs since the latter is a function that decreases with the memory size. Various efforts have been made at reducing the memory footprint of Strassen’s algorithm (cf. [11, 23, 35, 46, 47, 48, 49, 56, 57, 62]). In practical implementations, Bailey, Lee, and Simon [11], and many more [35, 46, 48, 49], obtained and utilized schedules of Strassen’s algorithm that reduce its memory footprint. Lai et al. [62], as well as [47, 57], addressed this challenge in the context of parallel implementations on GPUs, and Kumar et al. [58]

²See Algorithm 2.1.

³By linear operations we mean addition, subtraction, and multiplication by a fixed scalar.

TABLE 2
Communication costs.

Algorithm	Memory footprint	Communication costs
[86]	$3n^2$ [23]	$28.05 \left(\frac{n}{\sqrt{M}}\right)^{\log_2 7} M - 15n^2$
[55]	$(7\frac{1}{3} + o(1)) n^2$ [Naïve]	$33.52 \left(\frac{n}{\sqrt{M}}\right)^{\log_2 7} M - 6n^2 + 9n^2 \log_2 \left(\sqrt{2} \cdot \frac{n}{\sqrt{M}}\right)$
[55]	$3n^2$ [Here]	$23.37 \left(\frac{n}{\sqrt{M}}\right)^{\log_2 7} M - 6n^2 + 9n^2 \log_2 \left(\sqrt{2} \cdot \frac{n}{\sqrt{M}}\right)$
Here	$(2\frac{2}{3} + o(1)) n^2$ [Here]	$18.82 \left(\frac{n}{\sqrt{M}}\right)^{\log_2 7} M - 6.42n^2 + 3.25n^2 \log_2 \left(\sqrt{0.75} \cdot \frac{n}{\sqrt{M}}\right)$

minimized memory requirements in their parallelization of Strassen's algorithm on the Cray Y-MP.

Bailey, Lee, and Simon [11] suggested a schedule for Strassen's algorithm that reduces the extra memory requirements compared to the naïve schedule, at the expense of additional data movements. They used this schedule for the first few recursive steps and the naïve schedule for the rest.

Douglas et al. [35], and Huss-Lederman et al. [48, 49], suggested schedules for Strassen–Winograd's algorithm that use only 2 block matrices of size $\frac{n}{2} \times \frac{n}{2}$ each in each recursion level ($\frac{2}{3}n^2$ extra memory), in addition to the memory for two input matrices and an output matrix. They did so by reusing the additional memory blocks and output blocks for intermediate results.

Kreczmar [56] suggested storing the multiplication result in either the left side or the right side operand. While this schedule also uses 2 block matrices in each recursion level, the memory for the output matrix is not required. Unfortunately, this reduction comes at the expense of additional copy operations.

Boyer et al. [23] proposed two in-place schedules for Strassen–Winograd's algorithm that require no extra memory, other than for the two input matrices and the output matrix. They obtained the schedules using a brute force search that is similar to the corresponding pebbling game over the computation graph in [49]. They proposed one schedule that does not overwrite the input matrices but incurs an increase in the number of linear operations, and another schedule that maintains the original number of linear operations, by allowing overwriting of the input matrices.

Our contribution. We propose a new schedule for the bilinear phase of our algorithm, using a pebbling game. This schedule reduces the memory footprint to $(2\frac{2}{3} + o(1))n^2$ without incurring additional costs, and consequentially reduces the communication costs compared to [55] (see Table 2).

We prove that any invertible square basis transformation can be computed in place of the input matrix rather than using additional memory for an output matrix as in [55]. Therefore, we reduce the basis transformation's memory footprint and hence decrease communication costs.

1.4. Communication costs. For many matrix multiplication instances, the bottleneck of the runtime performance does not lie in the arithmetic complexity of the algorithm, rather in its communication costs, i.e., the number of read and write operations performed across the memory hierarchy and between cores or processors.

The communication costs lower bound for classical matrix multiplication algorithms is $\Omega\left(\frac{n^3}{PM^{1/2}} + \frac{n^2}{P^{2/3}}\right)$ [13, 50], for multiplying n -by- n matrices, using P processors with a local memory of size M words each. These bounds are attained by the 2D algorithms [25, 84] and the 3D algorithms [5, 20], for specific values of M , and by the

2.5D and the BFS-DFS algorithms [32, 59, 66, 76, 77], for an entire range of M values (up to a $\text{polylog}(P)$ factor).

The communication costs lower bound for fast matrix multiplication algorithms is $\Omega((\frac{n}{M^{1/2}})^{\omega_0} \frac{M}{P} + \frac{n^2}{P^{2/\omega_0}})$ [15, 16, 21, 68, 73], where $\Theta(n^{\omega_0})$ is the arithmetic complexity of the algorithm. This bound is not attained by all parallel implementations of Strassen's algorithm (cf. [33, 41, 65]). However some parallelizations of Strassen's algorithm, such as [14, 64, 66], do minimize the communication costs and attain these bounds up to a logarithmic factor in P . By applying Tang's approach [81] to [64], the communication costs can be minimized for an arbitrary number of processors, rather than for a multiple of a power of 7, as in [64]. In the sequential model (assigning $P = 1$ for the lower bounds), Strassen's algorithm attains the lower bound by using a block-recursive implementation.

Cohen and Roth [27], as well as [64, 82], presented data layouts for enhancing locality. Gepner, Gamayunov, and Fraser [40] discussed utilizing hardware capabilities such as advanced vector extensions. Ballard and Benson [19] presented different methods for implementing additions and subtractions, which can reduce the coefficients of the communication costs, and differ from each other in the number of reads and writes they require. The alternative basis method [17, 18, 55] reduces the leading coefficient of the communication costs, in addition to that of the arithmetic complexity.

Our contribution. To reduce the number of reads and writes while computing additions and subtractions, we merge loops of linear operations that operate on common block matrices, in both the bilinear phase and the basis transformations. This utilizes methods of [19] for implementing linear operations, and optimizes them to our algorithm. The combined effect of this and the decrease in memory footprint, results in a significant reduction in the communication costs of the algorithm (see Table 2).

We prove in Theorem 5.9 that given the same memory footprint, using the alternative basis method improves communication costs and arithmetic costs by the same ratio. This implies that the speedup factor is independent of whether the execution is computation or communication bound.

1.5. Paper organization. In section 2 we provide preliminaries and algorithm review. In section 3 we present the improvements in arithmetic complexity. In section 4 we show how we reduce memory footprint and in section 5 we optimize communication costs. In section 6 we present the implementation details, performance results, and numerical stability, and in section 7 we discuss current and future work.

2. Preliminaries. In this section, we provide preliminaries regarding recursive matrix multiplication algorithms and the alternative basis method.

2.1. Notations.

Notation 2.1. Let $t \in \mathbb{N}$. Denote by $[t]$ the set $[t] = \{1, \dots, t\}$.

Notation 2.2. Let R be a ring, and let $A \in R^{n \times m}$, where $n = n_0^l$, $m = m_0^l$ for some $n_0, m_0, l \in \mathbb{N}$. Denote by $A_{i,j}$ the (i, j) th block of A , of size $\frac{n}{n_0} \times \frac{m}{m_0}$. The row order vectorization of A is recursively defined by

$$\vec{A} = \left(\overrightarrow{A_{1,1}}, \dots, \overrightarrow{A_{1,m_0}}, \dots, \overrightarrow{A_{n_0,1}}, \dots, \overrightarrow{A_{n_0,m_0}} \right)^T.$$

Notation 2.3. Given a matrix A , denote the number of nonzero entries in A by $\text{nnz}(A) = |\{x \in A \mid x \neq 0\}|$, the number of nonsingleton entries in A by $\text{nns}(A) = |\{x \in A \mid x \notin \{0, 1, -1\}\}|$, and the number of rows in A by $\text{rows}(A)$.

Notation 2.4. Given a matrix A , denote $q_A = \text{nns}(A) + \text{nnz}(A) - \text{rows}(A)$.

2.2. Recursive bilinear algorithms. We denote recursive bilinear matrix multiplication algorithms by their base case.

Notation 2.5. Denote by an $\langle n_0, m_0, k_0; t \rangle$ -algorithm, where $n_0, m_0, k_0, t \in \mathbb{N}$, a recursive bilinear matrix multiplication algorithm that in its base case multiplies matrices of dimensions $n_0 \times m_0$ and $m_0 \times k_0$, using t scalar multiplications.

Any such algorithm can be used recursively to multiply matrices of dimensions $n \times m$ and $m \times k$, where $n = n_0^l$, $m = m_0^l$, $k = k_0^l$ for some $l \in \mathbb{N}$. This is accomplished by segmenting each of the input matrices into $n_0 \times m_0$, $m_0 \times k_0$ blocks of sizes $\frac{n}{n_0} \times \frac{m}{m_0}$, $\frac{m}{m_0} \times \frac{k}{k_0}$, respectively, and applying the algorithm on these blocks. Linear combinations are computed elementwise, while block multiplications are computed via recursive invocations of the algorithm. The asymptotic complexity of an $\langle n_0, n_0, n_0; t \rangle$ -algorithm is $O(n^{\omega_0})$, where $n = n_0^l$ for some $l \in \mathbb{N}$, and $\omega_0 = \log_{n_0} t$.

Fact 2.6. Bilinear representation. Let R be a ring, and let $n_0, m_0, k_0, t \in \mathbb{N}$. Let $f : R^{n_0 \cdot m_0} \times R^{m_0 \cdot k_0} \rightarrow R^{n_0 \cdot k_0}$ be a bilinear algorithm that performs t multiplications. Then there exist $U \in R^{t \times n_0 \cdot m_0}$, $V \in R^{t \times m_0 \cdot k_0}$, $W \in R^{t \times n_0 \cdot k_0}$ such that

$$\forall x \in R^{n_0 \cdot m_0}, y \in R^{m_0 \cdot k_0} : f(x, y) = W^T ((U \cdot \vec{x}) \odot (V \cdot \vec{y})),$$

where $\odot$ is the elementwise vector product (Hadamard product).

DEFINITION 2.7. *Encoding/decoding matrices.* Let $\langle U, V, W \rangle$ be the matrices representing a recursive bilinear algorithm. Then U, V are the encoding matrices and W is the decoding matrix.

CLAIM 2.8. *Additive complexity [18].* Let ALG be an $\langle n_0, m_0, k_0; t \rangle$ -algorithm represented by the encoding/decoding matrices $\langle U, V, W \rangle$. Then q_U, q_V, q_{W^T} are the number of linear operations incurred by applying each of U, V, W^T , respectively, and ALG performs $q = q_U + q_V + q_{W^T}$ linear operations at its base case.

CLAIM 2.9 (see [55]). Let ALG be an $\langle n_0, n_0, n_0; t \rangle$ -algorithm with q linear operations at its base case. The arithmetic complexity of ALG is

$$F_{ALG}(n) = \left(1 + \frac{q}{t - n_0^2}\right) n^{\omega_0} - \left(\frac{q}{t - n_0^2}\right) n^2.$$

2.3. Alternative basis recursive bilinear algorithms. The number of linear operations incurred by an $\langle n_0, m_0, k_0; t \rangle$ -algorithm can be reduced by transforming the basis of the input and output matrices to a representation in which the corresponding encoding/decoding matrices are sparser [55].

DEFINITION 2.10 (see [18, 55]). Let R be a ring, and let $\psi^1 : R^{n_0 \cdot m_0} \rightarrow R^{n_0 \cdot m_0}$ be an invertible linear map. We recursively define a linear map $\psi^{k+1} : R^{n \cdot m} \rightarrow R^{n \cdot m}$, where $n = n_0^l$, $m = m_0^l$ for some $k+1 \leq l$, applied to an input matrix $A \in R^{n \times m}$, by

$$\psi^{k+1}(\vec{A}) = \psi^1 \left(\psi^k(\vec{A}_1), \psi^k(\vec{A}_2), \dots, \psi^k(\vec{A}_{n_0 m_0}) \right)^T.$$

When obvious from context, we omit the superscript of ψ .

CLAIM 2.11. Let R be a ring and let $\psi \in R^{n_0 \cdot m_0 \times n_0 \cdot m_0}$ be a matrix representing an invertible linear map. Then ψ performs q_ψ linear operations at its base case, where q_ψ is as defined in Notation 2.4.

Proof. The proof is similar to the proof of Claim 2.8. □

Algorithm 2.1 Alternative basis matrix multiplication.

Input: $A \in R^{n \times m}$, $B \in R^{m \times k}$

Output: $C = A \cdot B$

1. **function** $ALG_{ABMM}(A, B)$:
 2. $\tilde{A} = \phi(\vec{A})$ ▷ Transform the basis of A
 3. $\tilde{B} = \psi(\vec{B})$ ▷ Transform the basis of B
 4. $\tilde{C} = ALG_{\phi, \psi, \nu}(\tilde{A}, \tilde{B})$ ▷ Compute a bilinear algorithm
 5. $\vec{C} = \nu^{-1}(\tilde{C})$ ▷ Transform the basis of C
 6. **return** C
-

CLAIM 2.12 (see [55]). Let R be a ring, let $\psi : R^{n_0 \cdot m_0} \rightarrow R^{n_0 \cdot m_0}$ be an invertible linear map, and let $A \in R^{n \times m}$, where $n = n_0^l$, $m = m_0^l$ for some $l \in \mathbb{N}$. The arithmetic complexity of computing $\psi(\vec{A})$ is

$$F_\psi(n, m) = \frac{q_\psi}{n_0 m_0} \cdot nm \log_{n_0 m_0}(nm).$$

DEFINITION 2.13 (see [18, 55]). Let R be a ring and let ϕ, ψ, ν be automorphisms over $R^{n_0 \cdot m_0}$, $R^{m_0 \cdot k_0}$, $R^{n_0 \cdot k_0}$, respectively. Let $A \in R^{n \times m}$, $B \in R^{m \times k}$, where $n = n_0^l$, $m = m_0^l$, $k = k_0^l$ for some $l \in \mathbb{N}$. Denote a recursive bilinear algorithm that takes $\phi(\vec{A})$, $\psi(\vec{B})$ as inputs and outputs $\nu(\vec{A} \cdot \vec{B})$, using t multiplications, by an $\langle n_0, m_0, k_0; t \rangle_{\phi, \psi, \nu}$ -algorithm.

CLAIM 2.14 (see [18, 55]). Let R be a ring and let ϕ, ψ, ν be automorphisms over $R^{n_0 \cdot m_0}$, $R^{m_0 \cdot k_0}$, $R^{n_0 \cdot k_0}$, respectively. $\langle U, V, W \rangle$ are the encoding/decoding matrices of an $\langle n_0, m_0, k_0; t \rangle_{\phi, \psi, \nu}$ -algorithm if and only if $\langle U \cdot \phi, V \cdot \psi, W \cdot \nu^{-T} \rangle$ are the encoding/decoding matrices of an $\langle n_0, m_0, k_0; t \rangle$ -algorithm.

Given an $\langle n_0, m_0, k_0; t \rangle_{\phi, \psi, \nu}$ -algorithm denoted by $ALG_{\phi, \psi, \nu}$, an alternative basis matrix multiplication algorithm ALG_{ABMM} is defined as in Algorithm 2.1.

COROLLARY 2.15 (see [55]). Let ALG_{ABMM} be an alternative basis matrix multiplication algorithm using an $\langle n_0, n_0, n_0; t \rangle_{\phi, \psi, \nu}$ -algorithm with q linear operations at its base case. The arithmetic complexity of ALG_{ABMM} is

$$F_{ALG_{ABMM}}(n) = \left(1 + \frac{q}{t - n_0^2}\right) n^{\omega_0} - \left(\frac{q}{t - n_0^2}\right) n^2 + O(n^2 \log_{n_0} n).$$

3. Arithmetic complexity. In this section, we present a new decomposition of Strassen's algorithm that reduces the number of linear operations incurred by each of the basis transformations while maintaining an optimal leading coefficient, and we analyze its arithmetic complexity.

3.1. Optimal $\langle 2, 2, 2; 7 \rangle$ -algorithm decomposition. We decompose Strassen's algorithm and obtain an alternative basis matrix multiplication algorithm that uses a $\langle 2, 2, 2; 7 \rangle_{\phi_{opt}, \psi_{opt}, \nu_{opt}}$ -algorithm. The basis transformations ϕ_{opt} , ν_{opt} and the encoding/decoding matrices $\langle U_{opt}, V_{opt}, W_{opt} \rangle$ representing the bilinear phase, are defined in (3.1). We prove the correctness of this decomposition in Appendix A. See Appendix B for the implied computations. This decomposition maintains an optimal leading coefficient as in [18, 55].

We refer to this algorithm as alternative basis Strassen and denote it by ABS .

$$(3.1) \quad \begin{aligned} U_{opt} &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & -1 & 0 & 1 \\ -1 & 0 & 0 & 1 \end{pmatrix}, V_{opt} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & -1 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix}, W_{opt} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & -1 & 0 & 0 \end{pmatrix}, \\ \phi_{opt} &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & -1 & 1 \end{pmatrix}, \nu_{opt}^{-1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & -1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \end{aligned}$$

3.2. Basis transformations. We analyze the arithmetic complexity of alternative basis Strassen's basis transformations.

LEMMA 3.1. *Each of ϕ_{opt} or ν_{opt} , defined in (3.1), performs $q_{\phi_{opt}} = q_{\nu_{opt}^{-1}} = 2$ linear operations at its base case.*

Proof. The results follow immediately by Claim 2.11. $\square$

COROLLARY 3.2. *Let R be a ring and let $A \in R^{n \times n}$, where $n = 2^l$ for some $l \in \mathbb{N}$. Let ϕ_{opt}, ν_{opt} be as defined in (3.1). The arithmetic complexity of applying $\phi_{opt}(\vec{A})$ or $\nu_{opt}^{-1}(\vec{A})$ is*

$$F_{\phi_{opt}}(n) = F_{\nu_{opt}^{-1}}(n) = 0.5 \cdot n^2 \log_2 n.$$

Proof. The results follow by Lemma 3.1 and Claim 2.12. $\square$

3.3. Bilinear algorithm. We next analyze the arithmetic complexity of alternative basis Strassen.

LEMMA 3.3. *The bilinear phase of alternative basis Strassen, as defined in (3.1), performs $q = 12$ linear operations at its base case.*

Proof. The result follows by Claim 2.8, since $q_{U_{opt}} = q_{V_{opt}} = 3$, $q_{W_{opt}^T} = 6$. $\square$

COROLLARY 3.4. *Let R be a ring, and let $A, B \in R^{n \times n}$, where $n = 2^l$ for some $l \in \mathbb{N}$. The arithmetic complexity of alternative basis Strassen, as defined in (3.1), is*

$$F_{ABS}(n) = 5n^{\log_2 7} - 4n^2 + 1.5n^2 \log_2 n.$$

Proof. By Lemma 3.3 it holds that $q = 12$. Therefore by Claim 2.9 the arithmetic complexity of the bilinear phase of alternative basis Strassen is $5n^{\log_2 7} - 4n^2$. When adding to this the arithmetic complexity of applying each of the basis transformations $\phi_{opt}, \phi_{opt}, \nu_{opt}^{-1}$ as analyzed in Corollary 3.2, we obtain the result. $\square$

The analysis for a shared memory machine with P processors is the same, up to a factor of $\frac{1}{P}$.

4. Memory footprint. In this section, we present the reduction in memory footprint of the basis transformations and of the bilinear phase of the algorithm.

4.1. Basis transformations. We prove that any invertible square basis transformation can be computed in place of the input matrix, as opposed to using additional memory for the transformed matrices, as in [55].

DEFINITION 4.1. Let R be a ring, and let $\psi : R^{n_0 \cdot m_0} \rightarrow R^{n_0 \cdot m_0}$ be an invertible linear map such that $\forall i \in [n_0 \cdot m_0]$, $\psi_{ii} \neq 0$. Then the directed graph $G_\psi = \langle V, E \rangle$ is defined by $V = \{v_1, \dots, v_{n_0 \cdot m_0}\}$ and $E = \{(v_j, v_i) \mid i \neq j, \psi_{ij} \neq 0\}$.

LEMMA 4.2. Let R be a ring and let $\langle U, V, W \rangle$ be the encoding/decoding matrices of an $\langle n_0, m_0, k_0; t \rangle_{\phi, \psi, \nu}$ -algorithm, with automorphisms ϕ, ψ, ν over $R^{n_0 \cdot m_0}, R^{m_0 \cdot k_0}, R^{n_0 \cdot k_0}$, respectively.

For any permutation matrix $P \in R^{n_0 \cdot m_0 \times n_0 \cdot m_0}$, $\langle U \cdot P^T, V, W \rangle$ are the encoding/decoding matrices of an $\langle n_0, m_0, k_0; t \rangle_{(P \cdot \phi), \psi, \nu}$ -algorithm.

For any permutation matrix $P \in R^{m_0 \cdot k_0 \times m_0 \cdot k_0}$, $\langle U, V \cdot P^T, W \rangle$ are the encoding/decoding matrices of an $\langle n_0, m_0, k_0; t \rangle_{\phi, (P \cdot \psi), \nu}$ -algorithm.

For any permutation matrix $P \in R^{n_0 \cdot k_0 \times n_0 \cdot k_0}$, $\langle U, V, W \cdot P \rangle$ are the encoding/decoding matrices of an $\langle n_0, m_0, k_0; t \rangle_{\phi, \psi, (\nu^{-1} \cdot P)^{-1}}$ -algorithm.

Proof. The proof is immediate from Claim 2.14 when $U' = U \cdot P^T$, $\phi' = P \cdot \phi$, when $V' = V \cdot P^T$, $\psi' = P \cdot \psi$, and when $W' = W \cdot P$, $\nu' = (\nu^{-1} \cdot P)^{-1}$. $\square$

CLAIM 4.3. Let R be a ring, let $\psi : R^{n_0 \cdot m_0} \rightarrow R^{n_0 \cdot m_0}$ be an invertible linear map, and let $A \in R^{n \times m}$, where $n = n_0^l$, $m = m_0^l$ for some $l \in \mathbb{N}$. Assume without loss of generality that $\forall i \in [n_0 \cdot m_0]$, $\psi_{ii} \neq 0$ (otherwise, perform corresponding permutations by Lemma 4.2). If the directed graph $G_\psi = \langle V, E \rangle$ implied by ψ has no cycles then $\psi(\vec{A})$ can be computed in place of $\vec{A}$ with no increase to arithmetic costs.

Proof. G_ψ has no cycles. Therefore, there exists a topological ordering of the vertices of G_ψ . That is, there exists an ordering in which $\forall i, j \in [n_0 \cdot m_0]$ such that $(v_j, v_i) \in E$, v_j appears before v_i . Denote this ordering by $v_{i_1}, \dots, v_{i_{n_0 \cdot m_0}}$.

The edges of G_ψ represent computation dependencies. Computing $(\psi \vec{A})_i$, $\forall i \in [n_0 \cdot m_0]$, implies computing a linear combination of $\vec{A}_i$ ($\psi_{ii} \neq 0$) with the elements $\{\vec{A}_j \mid i \neq j, \psi_{ij} \neq 0\}$. This can be computed in place with no extra costs, as long as $\forall j \in [n_0 \cdot m_0]$ such that $\psi_{ij} \neq 0$ it holds that $\vec{A}_j$ has not yet been updated.

Hence, performing the computations in reverse order to the topological ordering results in a well defined schedule for computing the basis transformation in place. $\square$

Remark 4.4. The condition in Claim 4.3 is not necessary. For example, the basis transformation ψ_{opt} in [55], can be computed in place, while $G_{\psi_{opt}}$ contains a cycle.

COROLLARY 4.5. Let R be a ring, let ϕ_{opt}, ν_{opt} , be as defined in (3.1). Then $\phi_{opt}, \nu_{opt}^{-1}$ can be computed in place with no increase to their arithmetic costs.

Proof. The directed graphs implied by each of $\phi_{opt}, \nu_{opt}^{-1}$ do not contain cycles, therefore the corollary follows by Claim 4.3. $\square$

CLAIM 4.6. Let R be a ring, let $\psi : R^{n_0 \cdot m_0} \rightarrow R^{n_0 \cdot m_0}$ be an invertible linear map, let $\psi_1, \dots, \psi_k \in R^{n_0 \cdot m_0 \times n_0 \cdot m_0}$ be elementary matrices such that $\psi = \psi_1 \cdot \dots \cdot \psi_k$, and let $A \in R^{n \times m}$, where $n = n_0^l$, $m = m_0^l$ for some $l \in \mathbb{N}$. Then $\psi(\vec{A})$ can be computed in place with at most k linear operations at the base case of ψ .

Proof. In each recursion level $\psi(\vec{A})$ may be computed by sequentially applying $\psi_k, \dots, \psi_1$. Hence it is sufficient to show that each ψ_i can be applied in place.

Let us assume without loss of generality that none of the elementary matrices correspond to the elementary row operation of swap one row with another (otherwise, perform corresponding permutations by Lemma 4.2). Therefore, each ψ_i corresponds to one of two elementary row operations: (1) Multiply a row by a nonzero scalar, i.e., multiply an input block by the scalar, or (2) add to a row a multiple of another row,

i.e., add to an input block a multiple of another. Each of these can be computed in place, requiring a single linear operation at the base case. $\square$

4.2. Bilinear algorithm. We reduce the memory footprint of the bilinear phase of the algorithm, by reusing memory blocks once they are no longer required. We obtain such a schedule by solving a variant of the pebbling game, suggested by [23, 49], on the algorithm's computation dependency graph.

The pebbling game consists of a directed acyclic graph (DAG) representing the computation dependencies of the algorithm on inputs of a given size, and of pebbles. The vertices represent the input blocks, output blocks, and all temporary blocks. There exists an edge from one vertex to another if and only if the first is an input of the computation resulting in the second. The pebbles each represent a required memory block, and can be placed on the vertices.

The game begins with a pebble on each input vertex and ends when all output vertices are covered with pebbles. The goal is to obtain this, using a minimal number of pebbles and following the rules:

1. A pebble can be removed from a vertex if all of its outgoing vertices are covered, indicating that all computations using this vertex were computed, thus its memory is no longer required.
2. A pebble can be placed on a vertex if all of its entering vertices are covered, indicating the execution of the computation. Such a pebble can either be a new one, or one that was previously removed from another vertex.
3. A pebble can be moved from a vertex u to another vertex v if all vertices entering v are covered, and all other outgoing vertices of u are covered, indicating the execution of v 's computation in place of the operand u .

We solve this pebbling game on the computation dependencies DAG (see Figure 1) for the bilinear phase base case of alternative basis Strassen, by exhaustive search. We allocate 10 pebbles (8 for the input blocks, and 2 additional memory blocks for each recursion level), and backtrack when an additional pebble is required. By noting in each step which computation is executed using which memory blocks, we obtain the schedule (see Algorithm 4.1). This schedule is then used for each recursion level.

We allow overwriting of the input matrices A, B , i.e., reusing the pebbles placed on their blocks. In contrast to [49], we allow computing the multiplications in place of the operand A , rather than in an additional output matrix. Since the multiplications are invoked recursively and overwrite both operands, we add a rule requiring that a multiplication be executed only after the values of its operands are no longer required. In contrast to [23], we do not enable copying of memory blocks.

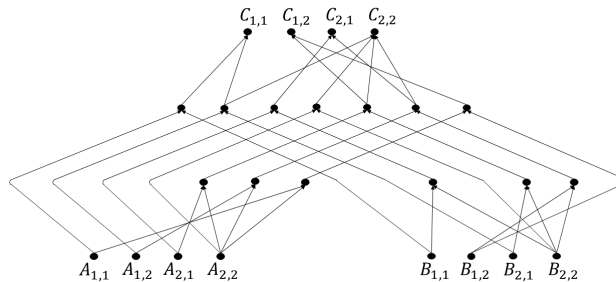


FIG. 1. The computation DAG of the bilinear phase of alternative basis Strassen.

Algorithm 4.1 Alternative basis Strassen bilinear phase.

$temp_1$	$=$	$-A_{1,1} + A_{2,2}$	$\triangleright$ Left input for m_7
$temp_2$	$=$	$-B_{1,1} + B_{2,2}$	$\triangleright$ Right input for m_3
$A_{1,1}$	$\cdot =$	$B_{1,1}$	$\triangleright m_1$
$B_{1,1}$	$=$	$-B_{1,2} + B_{2,2}$	$\triangleright$ Right input for m_6
$temp_1$	$\cdot =$	$B_{1,2}$	$\triangleright m_7$
$B_{1,2}$	$=$	$A_{2,1} + A_{2,2}$	$\triangleright$ Left input for m_5
$A_{2,1}$	$\cdot =$	$temp_2$	$\triangleright m_3$
$temp_2$	$=$	$B_{2,1} + B_{2,2}$	$\triangleright$ Right input for m_5
$B_{1,2}$	$\cdot =$	$temp_2$	$\triangleright m_5$
$temp_2$	$=$	$-A_{1,2} + A_{2,2}$	$\triangleright$ Left input for m_6
$A_{1,2}$	$\cdot =$	$B_{2,1}$	$\triangleright m_2$
$A_{2,2}$	$\cdot =$	$B_{2,2}$	$\triangleright m_4$
$temp_2$	$\cdot =$	$B_{1,1}$	$\triangleright m_6$
$A_{1,1}$	$+=$	$A_{1,2}$	$\triangleright$ Output block $C_{1,1} = m_1 + m_2$
$A_{2,1}$	$+=$	$temp_2$	$\triangleright$ Output block $C_{2,1} = m_3 + m_6$
$A_{2,2}$	$=$	$-A_{1,2} - A_{2,2}$	$\triangleright$ Output block $C_{2,2} = -m_2 - m_4$
$A_{2,2}$	$+=$	$B_{1,2}$	$\triangleright$ Output block $C_{2,2} += m_5$
$A_{2,2}$	$+=$	$temp_2$	$\triangleright$ Output block $C_{2,2} += m_6$
$A_{1,2}$	$=$	$B_{1,2} - temp_1$	$\triangleright$ Output block $C_{1,2} = m_5 - m_7$

CLAIM 4.7. Let ALG be an $\langle n_0, n_0, n_0; t \rangle$ -algorithm that given $A, B \in R^{n \times n}$, where $n = n_0^l$ for some $l \in \mathbb{N}$, outputs $A \cdot B$ in place of one of the operands. Assume ALG requires m additional block matrices in each recursion level. The memory footprint of ALG is

$$Mem_{ALG}(n) = \left[2 + \left(\frac{m}{n_0^2 - 1} \right) \right] n^2 - \left(\frac{m}{n_0^2 - 1} \right).$$

Proof. Each input matrix requires n^2 memory, and each step in the recursion requires m additional block matrices. Therefore

$$\begin{aligned} Mem_{ALG}(n) &= 2n^2 + \sum_{k=0}^{\log_{n_0} n - 1} m \left(\frac{n}{n_0^{k+1}} \right)^2 \\ &= 2n^2 + m \left(\frac{n}{n_0} \right)^2 \cdot \frac{\left(\frac{1}{n_0^2} \right)^{\log_{n_0} n} - 1}{\left(\frac{1}{n_0^2} \right) - 1}. \quad \square \end{aligned}$$

COROLLARY 4.8. The memory footprint for computing the bilinear phase of alternative basis Strassen, as defined in (3.1), is

$$Mem_{ABS}(n) = \left(2\frac{2}{3} + o(1) \right) n^2.$$

Proof. Algorithm 4.1 uses $m = 2$ additional block matrices in each recursion level. By Claim 4.7, $Mem_{ABS}(n) = 2\frac{2}{3}n^2 - \frac{2}{3}$. $\square$

5. Communication costs. The communication costs (IO-complexity) analysis corresponds to a 2-level memory hierarchy with fast memory of size M . For a shared memory machine with P processors, the same analysis holds up to a factor of $\frac{1}{P}$.

5.1. Basis transformations. In this section, we refine the communication costs analysis of the basis transformations, based on the reduction in memory footprint, and based on the fact that a basis transformation may not change or even use all input blocks, as opposed to the assumption that all input blocks need to be read into fast memory and written [55]. We define this and analyze the communication costs accordingly.

LEMMA 5.1. *Let R be a ring, let $\psi : R^{n_0 \cdot m_0} \rightarrow R^{n_0 \cdot m_0}$ be an invertible linear map that can be computed in place with no increase to arithmetic costs, and let $A \in R^{n \times m}$, where $n = n_0^l$, $m = m_0^l$ for some $l \in \mathbb{N}$. Computing $\psi(\vec{A})$ incurs reading and writing blocks. The indices of blocks to be read are $R_\psi = \{i \in [n_0 \cdot m_0] \mid \psi e_i \neq e_i \vee e_i^T \psi \neq e_i^T\}$, and the indices of blocks to be written are $W_\psi = \{i \in [n_0 \cdot m_0] \mid e_i^T \psi \neq e_i^T\}$.*

Proof. ψ is applied on $n_0 \cdot m_0$ blocks of $\vec{A}$. The i th block of $\vec{A}$ does not need to be written only if it is not changed by applying ψ , i.e., if $e_i^T \psi = e_i^T$. Therefore, $W_\psi = [n_0 \cdot m_0] \setminus \{i \in [n_0 \cdot m_0] \mid e_i^T \psi = e_i^T\}$.

The i th block of $\vec{A}$ does not need to be read only if it is not changed by applying ψ , i.e., if $e_i^T \psi = e_i^T$, and if it is not required for computing any other block of $\psi \vec{A}$, i.e., if $\psi e_i = e_i$. Therefore, $R_\psi = [n_0 \cdot m_0] \setminus \{i \in [n_0 \cdot m_0] \mid \psi e_i = e_i \wedge e_i^T \psi = e_i^T\}$. $\square$

LEMMA 5.2. *Let R be a ring, let $\psi : R^{n_0 \cdot m_0} \rightarrow R^{n_0 \cdot m_0}$ be an invertible linear map that can be computed in place with no increase to arithmetic costs, and let $A \in R^{n \times m}$, where $n = n_0^l$, $m = m_0^l$ for some $l \in \mathbb{N}$. Computing $\psi(\vec{A})$ requires at most $|R_\psi|$ blocks in memory.*

Proof. It holds from Lemma 5.1 that $W_\psi \subseteq R_\psi$. Hence a block that is written is necessarily read. Therefore, R_ψ consists of all indices of blocks required for applying ψ to an input, each of which is required in memory. $\square$

Remark 5.3. For simplicity, we assume the number of blocks required in memory to compute $\psi(\vec{A})$ is exactly $|R_\psi|$; though for some basis transformations, this may be reduced by reusing memory blocks, similarly to section 4.2.

COROLLARY 5.4. *Let ϕ_{opt} , ν_{opt} be as defined in (3.1). Then $|R_{\phi_{opt}}| = |R_{\nu_{opt}^{-1}}| = 3$, $|W_{\phi_{opt}}| = 1$, $|W_{\nu_{opt}^{-1}}| = 2$.*

Proof. By Lemma 5.1, $R_{\phi_{opt}} = R_{\nu_{opt}^{-1}} = \{2, 3, 4\}$, $W_{\phi_{opt}} = \{4\}$, $W_{\nu_{opt}^{-1}} = \{2, 3\}$. $\square$

CLAIM 5.5. *Let R be a ring, let $\psi : R^{n_0 \cdot m_0} \rightarrow R^{n_0 \cdot m_0}$ be a linear map that can be computed in place with no increase to arithmetic costs, and let $A \in R^{n \times m}$, where $n = n_0^l$, $m = m_0^l$ for some $l \in \mathbb{N}$. The IO-complexity of computing $\psi(\vec{A})$ is*

$$IO_\psi(n, m, M) \leq 3 \cdot \frac{q_\psi}{n_0 m_0} \cdot nm \log_{n_0 m_0} \left(\frac{|R_\psi|}{n_0 m_0} \cdot \frac{nm}{M} \right) + \left(\frac{|R_\psi| + |W_\psi|}{n_0 m_0} \right) nm.$$

Proof. Each step in the recursion consists of $n_0 m_0$ subproblems and performs q_ψ linear operations on the submatrices. Each linear operation requires 3 data transfers (reading two inputs and writing an output). The base case is when the problem fits entirely into the fast memory, namely, when $|R_\psi|$ submatrices fit into memory, $|R_\psi| \left(\frac{nm}{n_0 m_0} \right) \leq M$. Then, the IO-complexity consists of $|R_\psi|$ reads and $|W_\psi|$ writes of submatrices, altogether $|R_\psi| \left(\frac{nm}{n_0 m_0} \right) + |W_\psi| \left(\frac{nm}{n_0 m_0} \right)$. Therefore:

$$IO_\psi(n, m, M) \leq \begin{cases} n_0 m_0 \cdot IO_\psi\left(\frac{n}{n_0}, \frac{m}{m_0}, M\right) + 3q_\psi \cdot \frac{nm}{n_0 m_0} & \frac{|R_\psi|}{n_0 m_0} \cdot \frac{nm}{M} > 1, \\ \left(\frac{|R_\psi| + |W_\psi|}{n_0 m_0} \right) nm & \text{otherwise.} \end{cases}$$

Denote the recurrence depth by $r = \log_{n_0 m_0} \left(\frac{|R_\psi|}{n_0 m_0} \cdot \frac{nm}{M} \right)$. By solving the recurrence, we obtain the result

$$\begin{aligned} IO_\psi(n, m, M) &\leq \sum_{k=0}^{r-1} 3q_\psi \left(\frac{nm}{n_0^{k+1} m_0^{k+1}} \right) (n_0 m_0)^k + (n_0 m_0)^r \cdot \left(\frac{|R_\psi| + |W_\psi|}{n_0 m_0} \right) \cdot \frac{nm}{n_0^r m_0^r} \\ &= 3 \cdot \frac{q_\psi}{n_0 m_0} \cdot nm \cdot \sum_{k=0}^{r-1} 1 + \left(\frac{|R_\psi| + |W_\psi|}{n_0 m_0} \right) nm. \end{aligned} \quad \square$$

COROLLARY 5.6. *Let R be a ring, let $\psi : R^{n_0 \cdot m_0} \rightarrow R^{n_0 \cdot m_0}$ be a linear map, and let $A \in R^{n \times m}$, where $n = n_0^l$, $m = m_0^l$ for some $l \in \mathbb{N}$. The IO-complexity of computing $\psi(\vec{A})$ is*

$$IO_\psi(n, m, M) = O \left(nm \log_{n_0 m_0} \left(\frac{nm}{M} \right) \right).$$

5.2. Bilinear algorithm. In this section, we refine the communication costs analysis of the bilinear phase to account for the reduced memory footprint. We prove a theorem regarding the expected improvement ratios in both arithmetic and communication costs achieved by applying the alternative basis method.

We refer to the memory footprint of a recursive bilinear algorithm, including the memory for the input and output matrices.

CLAIM 5.7. *Let ALG be an $\langle n_0, n_0, n_0; t \rangle$ -algorithm with q linear operations at its base case and $Mem_{ALG}(n) = (a + o(1))n^2$. The IO-complexity of ALG is*

$$IO_{ALG}(n, M) \leq 3b \left(1 + \frac{q}{t - n_0^2} \right) \cdot \left(\frac{n}{\sqrt{M}} \right)^{\omega_0} M - 3 \left(\frac{q}{t - n_0^2} \right) n^2,$$

where $b = (a + o(1))^{\frac{\omega_0}{2} - 1}$.

Proof. Each step in the recursion consists of t subproblems and performs q linear operations on the submatrices. Each linear operation requires 3 data transfers (reading two inputs and writing an output). The base case is when the problem fits entirely into the fast memory, namely, when $Mem_{ALG}(n) = (a + o(1))n^2 \leq M$. Then, the IO-complexity is of reading both input matrices and writing the output matrix, altogether $3n^2 \leq \frac{3}{(a+o(1))} \cdot M$. Therefore

$$IO_{ALG}(n, M) \leq \begin{cases} t \cdot IO_{ALG} \left(\frac{n}{n_0}, M \right) + 3q \cdot \left(\frac{n}{n_0} \right)^2 & \sqrt{a + o(1)} \frac{n}{\sqrt{M}} > 1, \\ \frac{3}{(a+o(1))} \cdot M & \text{otherwise.} \end{cases}$$

Denote the recurrence depth by $r = \log_{n_0} \left(\sqrt{a + o(1)} \frac{n}{\sqrt{M}} \right)$. By solving the recurrence, we obtain the result

$$\begin{aligned} IO_{ALG}(n, M) &\leq \sum_{k=0}^{r-1} 3q \left(\frac{n}{n_0^{k+1}} \right)^2 t^k + t^r \cdot \frac{3}{(a + o(1))} \cdot M \\ &= 3q \left(\frac{n}{n_0} \right)^2 \cdot \sum_{k=0}^{r-1} \left(\frac{t}{n_0^2} \right)^k + (a + o(1))^{\frac{\log_{n_0} t}{2} - 1} \left(\frac{n}{\sqrt{M}} \right)^{\log_{n_0} t} \cdot 3M \\ &= 3q \left(\frac{n}{n_0} \right)^2 \cdot \frac{\left(\frac{t}{n_0^2} \right)^r - 1}{\left(\frac{t}{n_0^2} \right) - 1} + 3b \left(\frac{n}{\sqrt{M}} \right)^{\omega_0} M. \end{aligned} \quad \square$$

COROLLARY 5.8. *Let ALG_{ABMM} be an alternative basis matrix multiplication algorithm that uses an $\langle n_0, n_0, n_0; t \rangle_{\phi, \psi, \nu}$ -algorithm with q linear operations at its base case and $Mem(n) = (a + o(1))n^2$. The IO-complexity of ALG_{ABMM} is*

$$IO_{ALG_{ABMM}}(n, M) \leq 3b \left(1 + \frac{q}{t - n_0^2}\right) \cdot \left(\frac{n}{\sqrt{M}}\right)^{\omega_0} M - 3 \left(\frac{q}{t - n_0^2}\right) n^2 + O\left(n^2 \log_{n_0} \left(\frac{n}{\sqrt{M}}\right)\right),$$

where $b = (a + o(1))^{\frac{\omega_0}{2} - 1}$.

By applying the alternative basis method to a given algorithm ALG , to obtain a new alternative basis algorithm ALG_{ABMM} that performs fewer linear operations at its bilinear phase base case, then the improvement ratios in both arithmetic and communication costs are the same, given the same memory footprint.

THEOREM 5.9. *Let ALG be an $\langle n_0, n_0, n_0; t \rangle$ -algorithm that uses $Mem(n) = (a + o(1))n^2$, and let ALG_{ABMM} be the algorithm obtained by applying the alternative basis method to ALG , with $Mem_{ABMM}(n) = (a_{ABMM} + o(1))n^2$. Then⁴*

$$io_{ratio} = \left(\frac{a_{ABMM} + o(1)}{a + o(1)}\right)^{\frac{\omega_0}{2} - 1} \cdot f_{ratio},$$

where f_{ratio} , io_{ratio} are the ratios of the arithmetic complexities' and the IO-complexities' leading coefficients, respectively, between ALG_{ABMM} and ALG .

Proof. Denote by q , q_{ABMM} the number of linear operations at the base case of ALG , and at the bilinear phase base case of ALG_{ABMM} , respectively.

The leading coefficient of the arithmetic complexity of ALG is $(1 + \frac{q}{t - n_0^2})$ by Claim 2.9, and of ALG_{ABMM} is $(1 + \frac{q_{ABMM}}{t - n_0^2})$ by Corollary 2.15. The leading coefficient of the IO-complexity of ALG is $3(a + o(1))^{\frac{\omega_0}{2} - 1} (1 + \frac{q}{t - n_0^2})$ by Claim 5.7, and that of ALG_{ABMM} by Corollary 5.8 is $3(a_{ABMM} + o(1))^{\frac{\omega_0}{2} - 1} (1 + \frac{q_{ABMM}}{t - n_0^2})$. Therefore:

$$io_{ratio} = \frac{3(a_{ABMM} + o(1))^{\frac{\omega_0}{2} - 1} \left(1 + \frac{q_{ABMM}}{t - n_0^2}\right)}{3(a + o(1))^{\frac{\omega_0}{2} - 1} \left(1 + \frac{q}{t - n_0^2}\right)} = \left(\frac{a_{ABMM} + o(1)}{a + o(1)}\right)^{\frac{\omega_0}{2} - 1} \cdot f_{ratio}. \quad \square$$

Further reducing the number of linear operations performed by each basis transformation (as in section 3.1) leads to a similar result, namely, the IO-complexity is improved by the same ratio as the arithmetic complexity.

CLAIM 5.10. *Let R be a ring, let $\psi : R^{n_0 \cdot m_0} \rightarrow R^{n_0 \cdot m_0}$ be a linear map, and let $A \in R^{n \times m}$, where $n = n_0^l$, $m = m_0^l$ for some $l \in \mathbb{N}$. Let $\psi' : R^{n_0 \cdot m_0} \rightarrow R^{n_0 \cdot m_0}$ be a linear map that satisfies $q_{\psi'} < q_{\psi}$. Assume that ψ and ψ' can be computed in place with no increase to their arithmetic costs. Then*

$$f_{ratio} = \frac{q_{\psi'}}{q_{\psi}} = io_{ratio},$$

where f_{ratio} , io_{ratio} are the ratios of the arithmetic complexities' and the IO-complexities' leading coefficients, respectively, for computing $\psi'(\vec{A})$ and $\psi(\vec{A})$.

⁴The communication costs analysis gives an upper bound on the communication costs. It holds with equality, if n satisfies that $\exists l$ s.t. $(a + o(1)) \left(\frac{n}{n_0}\right)^2 = M$, namely, if after l recursive steps the problem fits exactly into the fast memory.

Proof. By Claim 2.12, the arithmetic complexity leading coefficients of computing $\psi(\vec{A})$, $\psi'(\vec{A})$, respectively, are $\frac{q_\psi}{n_0 m_0}$, $\frac{q_{\psi'}}{n_0 m_0}$. By Claim 5.5, the leading coefficient of the IO-complexity of computing $\psi(\vec{A})$ and $\psi'(\vec{A})$ are $3 \cdot \frac{q_\psi}{n_0 m_0}$, $3 \cdot \frac{q_{\psi'}}{n_0 m_0}$, respectively. Therefore

$$f_{ratio} = \frac{\frac{q_{\psi'}}{n_0 m_0}}{\frac{q_\psi}{n_0 m_0}} = \frac{q_{\psi'}}{q_\psi} = \frac{3 \cdot \frac{q_{\psi'}}{n_0 m_0}}{3 \cdot \frac{q_\psi}{n_0 m_0}} = io_{ratio}. \quad \square$$

The communication costs of both the basis transformations and the bilinear phase can be further reduced, below io_{ratio} , using additional methods. We discuss this in the following section.

5.3. Reducing communication costs. In this section, we further reduce the communication costs of both the basis transformations and the bilinear phase, by merging linear operations that operate on common blocks. Consequently, we perform fewer reads and writes, as opposed to requiring two reads for inputs and a write for an output, per linear operation. We tighten the communication costs analysis accordingly, and analyze the communication costs of alternative basis Strassen.

CLAIM 5.11. *Let R be a ring, let $\psi : R^{n_0 \cdot m_0} \rightarrow R^{n_0 \cdot m_0}$ be a linear map that can be computed in place with no increase to arithmetic costs, and let $A \in R^{n \times m}$, where $n = n_0^l$, $m = m_0^l$ for some $l \in \mathbb{N}$. Assume that all linear operations are computed together, i.e., each required block is read once and each changed block is written once. The IO-complexity of computing $\psi(\vec{A})$ is*

$$IO_\psi(n, m, M) \leq \left(\frac{|R_\psi| + |W_\psi|}{n_0 m_0} \right) nm \log_{n_0 m_0} \left(\frac{|R_\psi|}{n_0 m_0} \cdot \frac{nm}{M} \right) + \left(\frac{|R_\psi| + |W_\psi|}{n_0 m_0} \right) nm.$$

Proof. The proof is similar to the one of Claim 5.5 when in each recursion step, performing q_ψ linear operations requires $|R_\psi|$ reads and $|W_\psi|$ writes rather than $3q_\psi$ data transfers of submatrices. The base case remains the same. $\square$

COROLLARY 5.12. *Let R be a ring and let $A \in R^{n \times n}$, where $n = 2^l$ for some $l \in \mathbb{N}$. Let ϕ_{opt} , ν_{opt} be as defined in (3.1). Assume that all linear operations of each basis transformation are computed together. The IO-complexity of computing $\phi_{opt}(\vec{A})$ or $\nu_{opt}^{-1}(\vec{A})$ is*

$$IO_{\phi_{opt}}(n, M) \leq n^2 \log_2 \left(\sqrt{0.75} \cdot \frac{n}{\sqrt{M}} \right) + n^2,$$

$$IO_{\nu_{opt}^{-1}}(n, M) \leq 1.25 \cdot n^2 \log_2 \left(\sqrt{0.75} \cdot \frac{n}{\sqrt{M}} \right) + 1.25n^2.$$

Proof. By Corollary 4.5 ϕ_{opt} , ν_{opt}^{-1} can be computed in place with no increase to their arithmetic costs. Hence the results follow by Corollary 5.4 and by Claim 5.11. $\square$

CLAIM 5.13. *Let ALG be an $\langle n_0, n_0, n_0; t \rangle$ -algorithm with q linear operations at its base case and $Mem_{ALG}(n) = (a + o(1))n^2$. Assume $io_q \leq 3q$ read and write operations are required for computing the linear operations. The IO-complexity of ALG is*

$$IO_{ALG}(n, M) \leq b \left(3 + \frac{io_q}{t - n_0^2} \right) \cdot \left(\frac{n}{\sqrt{M}} \right)^{\omega_0} M - \left(\frac{io_q}{t - n_0^2} \right) n^2,$$

where $b = (a + o(1))^{\frac{\omega_0}{2} - 1}$.

Proof. The proof is similar to the one of Claim 5.7, when in each recursion step, performing q linear operations requires io_q data transfers (reads and writes) of the submatrices, rather than $3q$. The base case remains the same. $\square$

COROLLARY 5.14. *Let R be a ring, and let $A, B \in R^{n \times n}$, where $n = 2^l$ for some $l \in \mathbb{N}$. The IO-complexity of alternative basis Strassen, as defined in (3.1), is*

$$IO_{ABS}(n, M) \leq 18.82 \left(\frac{n}{\sqrt{M}} \right)^{\log_2 7} M - 6.42n^2 + 3.25n^2 \log_2 \left(\sqrt{0.75} \cdot \frac{n}{\sqrt{M}} \right).$$

Proof. Algorithm 4.1 can be computed using $io_q = 29$ read and write operations. From Corollary 4.8 it holds that $Mem_{ABS}(n) = (2\frac{2}{3} + o(1))n^2$. Therefore by Claim 5.13 the IO-complexity of the bilinear phase of alternative basis Strassen is $18.82(\frac{n}{\sqrt{M}})^{\log_2 7} M - 9.67n^2$. When adding to this the communication costs of applying each of the basis transformations ϕ_{opt} , ϕ_{opt} , ν_{opt}^{-1} as analyzed in Corollary 5.12, we obtain the result. $\square$

6. Implementation and performance. In this section, we benchmark the runtimes of three algorithms: Intel's MKL DGEMM, Strassen–Winograd, and ours. We present the experimental setup, resulting speedups, and numerical stability.

6.1. Implementation details. Our algorithm was implemented in C++17. We computed the new basis transformations (see (3.1)) in place, and we implemented the bilinear phase according to the new schedule which reduces the memory footprint (see Algorithm 4.1). The number of reads and writes were reduced by merging linear operation loops (recall section 5.3). All loops that compute additions and subtractions were fully vectorized, that is, utilize SIMD intrinsics.

When reaching sufficiently small blocks, we invoked Intel's MKL DGEMM. That is, for multiplying matrices of dimensions $b \cdot 2^l \times b \cdot 2^l$, we ran alternative basis Strassen for l recursive steps and then invoked DGEMM on block matrices of dimensions $b \times b$.

We used a block-recursive layout for the matrices, also known as recursive Morton ordering [9, 37, 39, 87]. Each matrix was divided into four blocks that were placed one after the other in memory, in row-major order. This was applied recursively until reaching blocks of dimension $b \times b$ each, which were in row-major order.

We implemented a parallel variant of our algorithm as well, for a shared memory architecture. All loops that compute linear operations were parallelized using OpenMP directives, and when reaching small enough blocks, Intel's MKL DGEMM was invoked using all available threads. We used the same data layout for both the parallel version, and the sequential version.

For comparison, we implemented Strassen–Winograd's algorithm [86], as well. The algorithm was implemented according to the schedule presented by Boyer et al. [23] that minimizes the memory footprint to $3n^2$ without incurring additional costs. We used the same layout as described for our algorithm, composed the algorithm with DGEMM, and fully vectorized all loops that compute additions and subtractions.

6.2. Experimental setup. Our experiments were conducted on a dual-socket Intel's Xeon E5-2680 v3 processor, and on a dual-socket Intel Xeon Gold 6248 processor. The specifications of the processors are given in Table 3.

The code was compiled with `-O3 -march=native` flags, and the processor frequency was set to the base frequency throughout the experiments. We measured the runtimes of the algorithms when benchmarked on these machines, when multiplying square matrices with random double-precision elements. Intel's MKL DGEMM

TABLE 3
Benchmark specifications.

	Intel Xeon E5-2680	Intel Xeon Gold 6248
Microarchitecture	Haswell	Cascade Lake
Base frequency	2.50 GHz	2.50GHz
Number of cores per socket	12	20
Hyperthreads per core	2	1
SIMD vector width	256 bit	512 bit
L1d cache per core	Size: 32 KB Associativity: 8 Cache line size: 64	Size: 32 KB Associativity: 8 Cache line size: 64
L2 cache per core	Size: 256 KB Associativity: 8 Cache line size: 64	Size: 1024 KB Associativity: 16 Cache line size: 64
L3 cache per socket	Size: 30720 KB Associativity: 20 Cache line size: 64	Size: 28160 KB Associativity: 11 Cache line size: 64
Memory	DDR4 - 2133 Mhz 4 channels	DDR4 - 2933 Mhz 6 channels
Memory bandwidth	68 GB/s	140.8 GB/s
Compiler	GCC 8.3.0	GCC 11.2.0
Intel MKL version	2021.3.0	2022.1.0

uses row-major layout for the input and output matrices. In contrast, alternative basis Strassen, and Strassen–Winograd’s algorithm use a block-recursive layout (as described in section 6.1). We show in Figure 7(a) that the time required for converting the data layouts is negligible for large matrix dimensions. The runtime of alternative basis Strassen includes both the time required for all basis transformations and for the bilinear phase. For each matrix dimension we invoked the algorithm with a range of values of l (the number of recursive steps before switching to DGEMM), and chose the minimizing value.⁵

For each input size we took the average of three measurements when running sequentially, and the median of 30 measurements when running in parallel. Each measurement was conducted independently and with different matrix values to eliminate the effect of caching. We assume that the matrices are in the RAM when starting the computations, regardless of matrix dimension. The standard deviation was less than 1% for all sequential measurements, and less than 5% for all parallel measurements. For small input sizes, the runtime is subject to variations. We followed Intel’s recommendations [1] and measured each of the runs as an average of multiple iterations. Since the first invocation of MKL DGEMM performs initializations which require additional time, we invoked DGEMM once before measuring the results [1].

6.3. Runtime performance. In this section, we present performance results compared to Intel’s MKL DGEMM, benchmarked on both processors.

Sequential performance. Figure 2 presents the speedups of our algorithm compared to DGEMM for matrix dimension $n = 1024$ through $n = 46080$ in steps of 1024, benchmarked on Intel Xeon E5-2680 (2(a)) and on Intel Xeon Gold 6248 (2(b)). For Intel Xeon E5-2680, the speedups ranged from over 10% for $n = 2048$, up to 48.6% (speedup of $\times 1.946$) for $n = 46080$. The optimal block dimension sent to DGEMM ranged between $b = 224$ and $b = 512$, which fit into the L3 cache. Hence for optimal

⁵This process can be automated and performed offline, prior to the algorithm execution.

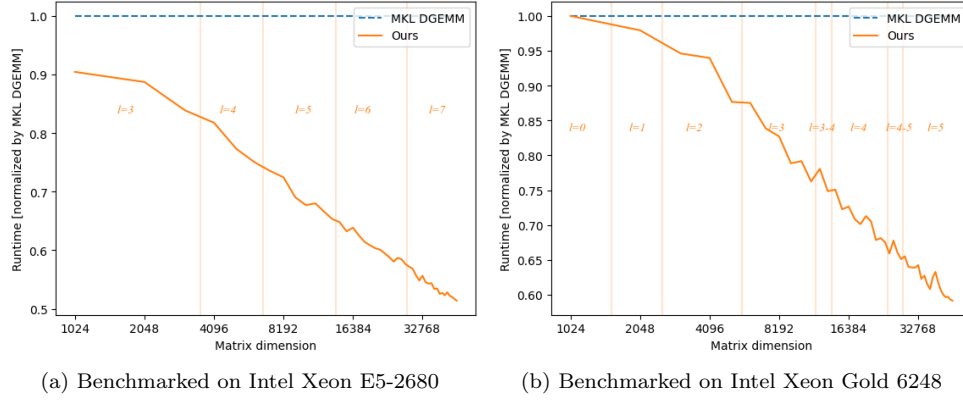


FIG. 2. Sequential improvement compared to DGEMM. The l values on the figures correspond to the optimal number of steps.

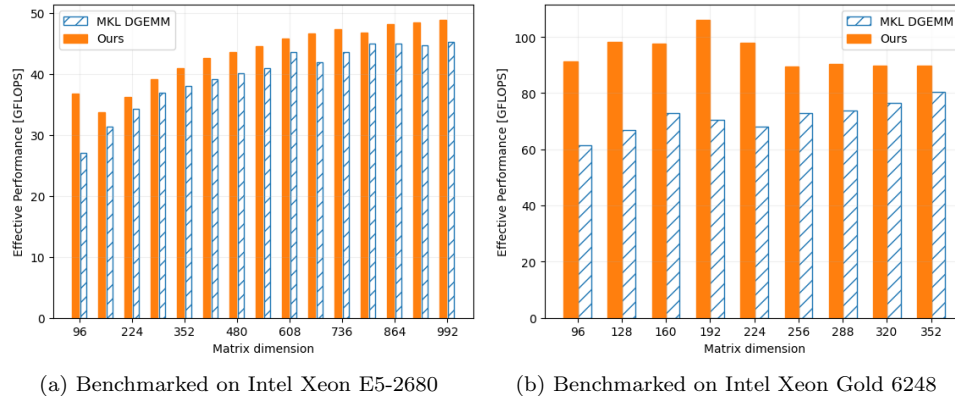


FIG. 3. Sequential improvement compared to DGEMM, $l = 1$.

block choice, our algorithm handles the communication between the L3 cache and the RAM, while DGEMM handles the communication within the rest of the cache hierarchy. When benchmarked on Intel Xeon Gold 6248, our algorithm tops DGEMM starting at $n = 2048$, with an increasing speedup of up to 40.84% (speedup of $\times 1.69$) for $n = 46080$. The optimal block dimension sent to DGEMM was larger compared to that measured on Intel Xeon E5-2680, and ranged between $b = 768$ and $b = 1728$.

Figure 3 depicts the speedups of our algorithm compared to DGEMM for small matrix dimensions, starting at matrix dimension $n = 96$, in terms of effective performance,⁶ defined by

$$\text{Effective Performance [GFLOP]} = \frac{2n^3}{\text{Average Runtime [seconds]}} \cdot 10^{-9}$$

In Figure 3(a) we present the results for small matrix dimensions, in steps of 64, benchmarked on Intel Xeon E5-2680, and using $l = 1$ recursion steps. Our algorithm

⁶This is a common measure for comparing performance of matrix multiplication algorithms. For a classical matrix multiplication algorithm it measures the exact flop rate, while for the fast matrix multiplication algorithm it measures flop rate normalized by $O(n^{3-\log_2 7})$.

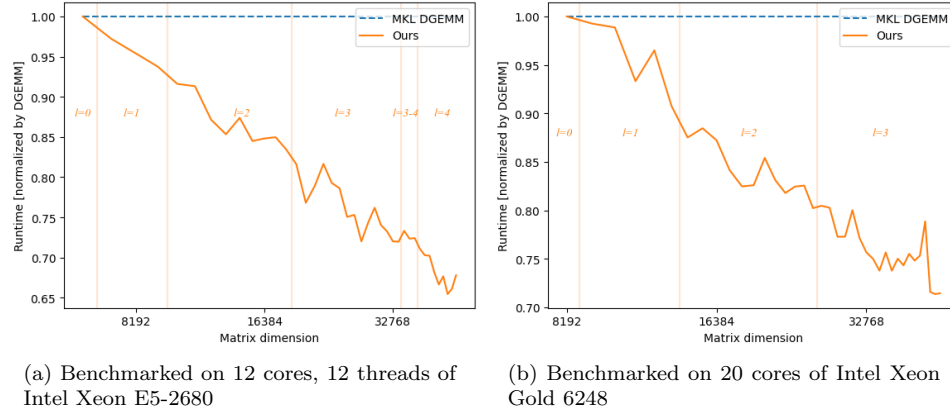


FIG. 4. Parallel improvement compared to DGEMM. The l values on the figures correspond to the optimal number of steps.

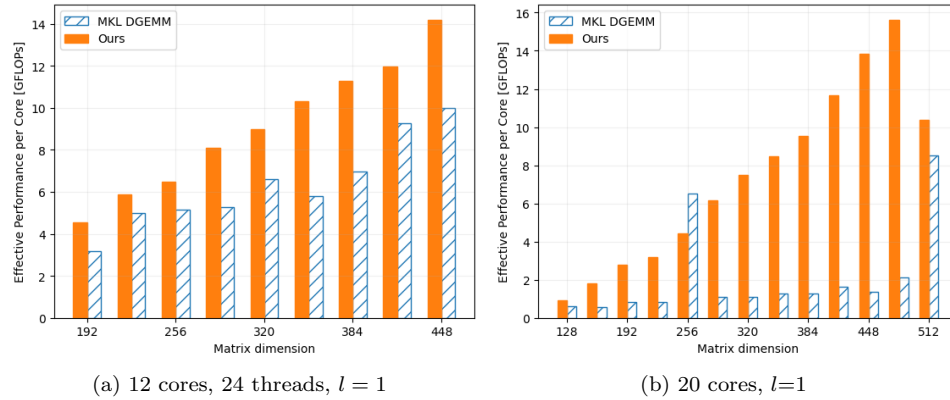


FIG. 5. Parallel improvement compared to DGEMM, $l = 1$.

outperforms DGEMM starting at $n = 96$ with a speedup of 26.41%. Figure 3(b) presents the results benchmarked on Intel Xeon Gold 6248, for small matrix dimensions, in steps of 32. Using $l = 1$ recursion steps, our algorithm obtained speedups compared to DGEMM, starting at matrix dimension $n = 96$, of up to 33.67% for $n = 192$. For matrices larger than $n = 352$ (up to $n = 2048$) DGEMM mostly obtained better performance compared to our algorithm.

Parallel performance. In Figure 4 we present the speedups obtained by our algorithm compared to DGEMM when running on 12 cores of a single socket of Intel Xeon E5-2680 (4(a)) and when running on 20 cores of a single socket of Intel Xeon Gold 6248 (4(b)). Figure 5 depicts the results for small matrix dimensions. For large matrix dimensions, on Intel Xeon E5-2680, our algorithm outperforms DGEMM starting at $n = 7168$, with an improvement of up to 34.53% for $n = 44032$. On Intel Xeon Gold 6248, our algorithm tops DGEMM starting at matrix dimension $n = 9126$, with a speedup of up to $\times 1.4$ for $n = 40680$.

When running the algorithm with 24 threads of Intel Xeon E5-2680 (single socket, using hyperthreading) our algorithm outperforms DGEMM for small matrices as well, demonstrating a speedup of 43.59% for $n = 352$ (see Figure 5(a)). When measuring

the runtime of our algorithm compared to DGEMM on Intel Xeon Gold 6248, for matrices of dimensions $n = 128$ up to $n = 512$, our algorithm obtains significant speedups compared to DGEMM, of up to 90.07% for $n = 448$. For small matrix dimensions the runtime of DGEMM varies by the number of cores and does not obtain perfect strong scaling. DGEMM achieved best runtime for $n = 448$, when running on 13 cores.

6.4. Effect of optimizations. In this section we analyze the effect of the optimizations on runtime performance.

Memory footprint reduction, loop merging. Figure 6(a) demonstrates the impact of the optimizations used, loop merging and memory footprint reduction, in terms of effective performance. The term memory footprint reduction refers to both performing the basis transformations in place, and reducing the memory footprint of the bilinear phase by rescheduling the algorithm. We benchmarked the effect of four versions of our algorithm (with and without memory footprint reduction, and with and without loop merging) on Intel Xeon Gold 6248. The results show that the reduction in memory footprint has a larger impact on performance, compared to loop merging. Note that there exists a schedule that reduces the leading coefficient of the communication costs further, by using 3 extra block matrices in each recursion level (rather than 2) and $io_q = 27$ (rather than $io_q = 29$ in our implementation, or $io_q = 36$ in [55]). However, this has a negligible effect of increasing runtime for the matrix dimensions measured, due to an increase in the low order coefficients, namely, the coefficients of $O(n^2)$.

Comparison to other algorithms. Figure 6(b) presents the runtime of our algorithm compared to our high performance implementation of Karstadt and Schwartz's algorithm [55] (previous implementation was found to be too slow), and of Ballard and Benson's implementation of Strassen's algorithm [19]. The improvements introduced to the coefficients and low order monomials have a large impact on small matrix dimensions. The runtime of our algorithm and of Karstadt and Schwartz's algorithm [55] is broken into the time required for basis transformations and the time required for the bilinear phase of the algorithm (including the time spent in DGEMM). For small matrix dimensions, our algorithm requires no longer that 12% of the runtime to perform basis transformations (compared to 15 – 48%, for Karstadt and Schwartz's algorithm). Figure 6(c) visualizes the improvements achieved by using our algorithm compared to Strassen–Winograd's algorithm for a matrix of dimension $n = 32768$, as a function of l (the recursive depth), demonstrating an increase in speedup as a function

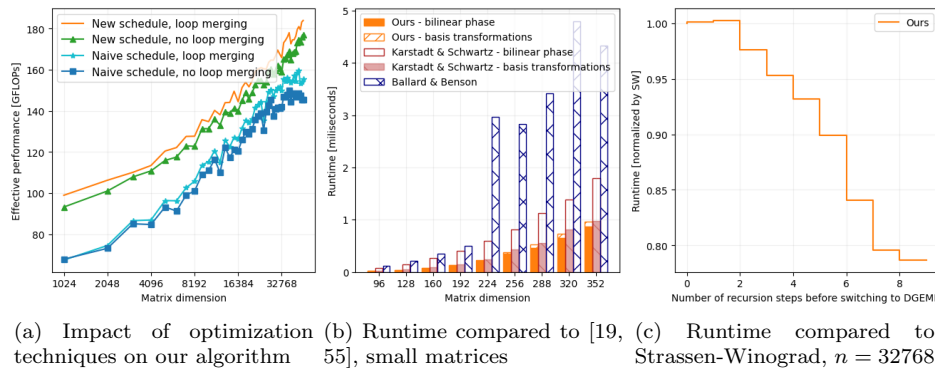
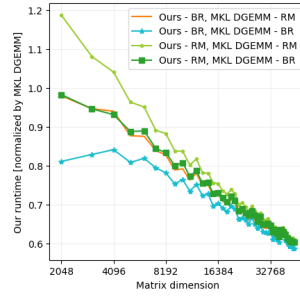


FIG. 6. Effect of optimizations, benchmarked on a single core of Intel Xeon Gold 6248.



(a) Improvement compared to DGEMM, using block-recursive (BR) and row-major (RM) data layouts

FIG. 7. Single core of Intel Xeon Gold 6248.

of l . As l increases, a larger fraction of the runtime is spent in the outer algorithm (either ours, or Strassen–Winograd’s algorithm), as opposed to in DGEMM, resulting in a decrease in runtime.

Data layout. In Figure 7(a) we present the improvement of our algorithm compared to DGEMM when both DGEMM and our algorithm start and end in the same data layout: row-major or block-recursive. The runtime of our algorithm in row-major includes both the time required to copy the memory of the input matrices, and to convert the data layouts into block recursive (inputs) and back to row major (output). The additional memory required for copying the matrices does not impact the communication costs of the algorithm since once the data are copied into the new layout, the bilinear phase uses the new layout only in the cache hierarchy (requiring $(2\frac{2}{3} + o(1))n^2$ memory). Consequentially, even when our algorithm uses inputs and outputs with suboptimal data layout and our algorithm is required to transform the layout, the speedups for larger matrix dimensions are maintained.

6.5. Numerical Stability. To measure the numerical stability of an algorithm, we use the measure max-norm error. It is defined by $\|C - \hat{C}\| = \max_{i,j} |C_{i,j} - \hat{C}_{i,j}|$, where C is the result of the algorithm and $\hat{C}$ is the exact result by the classical algorithm in quadruple precision, both when multiplying the same square matrices with elements drawn uniformly at random from $[-1, 1]$. We leave testing of numerical stability with elements drawn from other ranges for future research.

Figure 8(a) presents the max-norm error of the algorithms for a matrix of dimension $n = 32768$, as a function of l . The numerical stability of our algorithm does not differ from that of Strassen–Winograd’s algorithm. For $l = 0$, we present the max-norm error obtained by DGEMM. The error grows exponentially with l . For the optimal value $l = 7$, both Strassen–Winograd’s algorithm and ours lose approximately three decimal precision digits compared to DGEMM. The max-norm error of the algorithms for a range of smaller matrix dimensions are presented in Figure 8(b). Both Strassen–Winograd’s algorithm and ours lose less than one decimal precision digit compared to DGEMM, for small matrix dimensions, and approximately 2.5 decimal precision digits, for medium to large matrix dimensions.

7. Discussion and future work. Our implementation of fast matrix multiplication on CPUs obtains significant speedups compared to Intel’s MKL DGEMM.

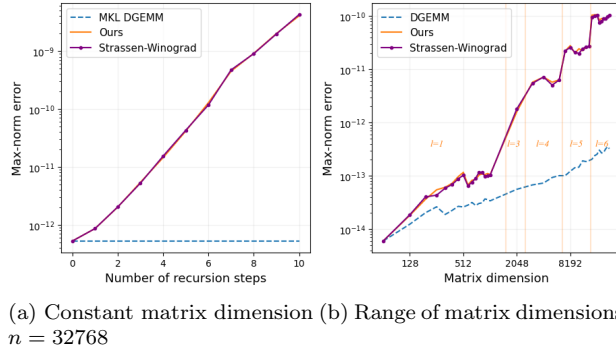


FIG. 8. Numerical stability benchmarked on Intel Xeon E5-2680 compared to classical matrix multiplication with quadruple precision.

We present and utilize several optimization methods, for reducing arithmetic computations, memory footprint, and communication costs, and apply them to Strassen's $\langle 2, 2, 2; 7 \rangle$ -algorithm.

Our algorithm attains the lower bound on communication costs, and further reduces the communication costs coefficients by constant factors, compared to [55].

Additional algorithms. The optimization methods we presented can be applied to various recursive bilinear algorithms, consequentially enhancing their practicality and performance. Potential candidates include algorithms with exponent smaller than $\log_2(7) \approx 2.807$ and small leading coefficients. For example, Smirnov's $\langle 6, 3, 3; 40 \rangle$ -algorithm [74] has an exponent of ≈ 2.774 and a leading coefficient that can be reduced to 8.9 [17], and Pan's $\langle 44, 44, 44; 36133 \rangle$ -algorithm [71] has an exponent of ≈ 2.773 and a leading coefficient that can be reduced to 8.08 [42]. These may offer better performance for sufficiently large matrices utilizing our optimization techniques.

Some algorithms have an exponent that is larger than Strassen's, but smaller leading coefficients (cf. [17, 19, 44, 61]), hence, have the potential to outperform Strassen's algorithm for a certain range of matrix dimensions. For example, Ballard and Benson's $\langle 3, 3, 3; 23 \rangle$ -algorithm [19] has a leading coefficient 2 when fully decomposed [18], the same as classical matrix multiplication algorithms. Other algorithms (cf. [38, 51, 60]), have an exponent larger than Strassen's, but feasible base case dimensions, and have not been decomposed yet.

Arbitrary square matrix dimensions. Our algorithm is currently designed to multiply matrices of dimensions $b \cdot 2^l \times b \cdot 2^l$, where the blocks of dimensions $b \times b$ are multiplied by invoking Intel's MKL DGEMM, and l is the number of recursive steps of the algorithm taken before invoking DGEMM of the inner algorithm. Alternatively, padding, peeling, and overlapping techniques may be used (c.f. [35, 48, 49, 82]).

While we presented results for the square case only, the algorithm can be used to multiply matrices of arbitrary rectangular dimensions, by adjusting the block dimensions and using a non-in-place schedule. Further, other fast matrix multiplication algorithms with corresponding rectangular base cases can be used, utilizing the same optimization methods. The algorithm can also be used as is for rectangular matrices, by dividing each matrix into square matrices.

In future work, we plan to implement and benchmark our algorithm on various hardware, such as GPU and hardware accelerators of matrix multiplication, and to

utilize multicore and multithreads for large matrix dimensions. We plan to examine techniques for improving our algorithm's numerical stability, such as diagonal scaling [12, 36, 43], and to apply our optimization methods to further bilinear algorithms.

Appendix A. Decomposition correctness. We prove the correctness of our decomposition of the $\langle 2, 2, 2; 7 \rangle$ -algorithm, presented in section 3.1.

Notation A.1. Let R be a ring, let $A \in R^{n_0 \times m_0}$, and let $U \in R^{t \times n_0 \cdot m_0}$ for $n_0, m_0, t \in \mathbb{N}$. Denote by $U_{l,(i,j)}$ the element in the l th row of U , in the column corresponding to the index of $A_{i,j}$ in $\vec{A}$.

Fact A.2. Triple product condition [24]. Let R be a ring, and let $U \in R^{t \times n_0 \cdot m_0}$, $V \in R^{t \times m_0 \cdot k_0}$, $W \in R^{t \times n_0 \cdot k_0}$ for $n_0, m_0, k_0, t \in \mathbb{N}$. Then $\langle U, V, W \rangle$ are the encoding/decoding matrices of an $\langle n_0, m_0, k_0; t \rangle$ -algorithm if and only if $\forall i_1, i_2 \in [n_0]$, $\forall l_1, l_2 \in [m_0]$, $\forall j_1, j_2 \in [k_0]$ it holds that

$$\sum_{r=1}^t U_{r,(i_1,l_1)} V_{r,(l_2,j_1)} W_{r,(i_2,j_2)} = \delta_{i_1,i_2} \delta_{l_1,l_2} \delta_{j_1,j_2},$$

where $\delta_{i,j} = 1$ if $i = j$ and 0 otherwise.

CLAIM A.3. $\langle U_{opt}, V_{opt}, W_{opt} \rangle$, as defined in (3.1), are the encoding/decoding matrices of a $\langle 2, 2, 2; 7 \rangle_{\phi_{opt}, \phi_{opt}, \nu_{opt}}$ -algorithm.

Proof. $\langle U_{opt} \cdot \phi_{opt}, V_{opt} \cdot \phi_{opt}, W_{opt} \cdot \nu_{opt}^{-T} \rangle$ satisfy the triple product condition in Fact A.2, thus are the encoding/decoding matrices of a $\langle 2, 2, 2; 7 \rangle$ -algorithm. Hence the claim follows by Claim 2.14. $\square$

Therefore alternative basis Strassen is a $\langle 2, 2, 2; 7 \rangle$ -algorithm.

Appendix B. Algorithm computations. We present the computations implied by each of the stages of alternative basis Strassen.

The basis transformation ϕ_{opt} of the input matrices A, B can be computed as follows, using 2 additions/subtractions each:

$$\phi_{opt}(\vec{A}) = \begin{pmatrix} A_{1,1} \\ A_{1,2} \\ A_{2,1} \\ A_{1,2} - A_{2,1} + A_{2,2} \end{pmatrix}, \quad \phi_{opt}(\vec{B}) = \begin{pmatrix} B_{1,1} \\ B_{1,2} \\ B_{2,1} \\ B_{1,2} - B_{2,1} + B_{2,2} \end{pmatrix}.$$

The bilinear phase of the algorithm can be computed on the transformed matrices, according to a naïve schedule as follows, using 7 multiplications and 12 additions:

$$\begin{array}{lll} temp_1 = A_{2,1} + A_{2,2}, & m_1 = A_{1,1} \cdot B_{1,1}, & C_{1,1} = m_1 + m_2, \\ temp_2 = A_{2,2} - A_{1,2}, & m_2 = A_{1,2} \cdot B_{2,1}, & C_{1,2} = m_5 - m_7, \\ temp_3 = A_{2,2} - A_{1,1}, & m_3 = A_{2,1} \cdot temp_4, & C_{2,1} = m_3 + m_6, \\ temp_4 = B_{2,2} - B_{1,1}, & m_4 = A_{2,2} \cdot B_{2,2}, & C_{2,2} = m_5 + m_6, \\ temp_5 = B_{2,1} + B_{2,2}, & m_5 = temp_1 \cdot temp_5, & C_{2,2} = m_2, \\ temp_6 = B_{2,2} - B_{1,2}, & m_6 = temp_2 \cdot temp_6, & C_{2,2} = m_4, \\ & m_7 = temp_3 \cdot B_{1,2}, & \end{array}$$

The inverse basis transformation ν_{opt}^{-1} of the output matrix C can be computed as follows, using 2 additions/subtractions:

$$\nu_{opt}^{-1}(\vec{C}) = \begin{pmatrix} C_{1,1} \\ C_{1,2} - C_{2,2} \\ -C_{2,1} + C_{2,2} \\ C_{2,2} \end{pmatrix}.$$

REFERENCES

- [1] *Tips to Measure the Performance of Matrix Multiplication using Intel® MKL*, <https://www.intel.com/content/www/us/en/developer/articles/technical/a-simple-example-to-measure-the-performance-of-an-intel-mkl-function.html> (2017).
- [2] *Intel® oneAPI Math Kernel Library*, <https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html> (2020).
- [3] *AMD Optimizing CPU Libraries (AOCL)*, <https://developer.amd.com/amd-aocl/#documentation1> (2022).
- [4] *Nvidia cuBLAS*, <https://developer.nvidia.com/cublas> (2022).
- [5] R. C. AGARWAL, S. M. BALLE, F. G. GUSTAVSON, M. JOSHI, AND P. PALKAR, *A three-dimensional approach to parallel matrix multiplication*, IBM J. Res. Dev., 39 (1995), pp. 575–582.
- [6] M. AL-MOUHAMED, A. FATAYER, N. MOHAMMAD, A. ul Hassan Khan, *Optimizing the matrix multiplication using Strassen and Winograd algorithms with limited recursions on many-core*, Int. J. Parallel Program., 44 (2016), pp. 801–830.
- [7] J. ALMAN AND V. V. WILLIAMS, *A refined laser method and faster matrix multiplication*, in Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA), SIAM, Philadelphia, 2021, pp. 522–539.
- [8] E. ANDERSON, Z. BAI, C. BISCHOF, S. BLACKFORD, J. DEMMEL, J. DONGARRA, J. DU CROZ, A. GREENBAUM, S. HAMMARLING, A. MCKENNEY, AND D. SORENSSEN, *LAPACK Users' Guide*, 3rd, ed., SIAM, Philadelphia, 1999.
- [9] M. BADER, R. FRANZ, S. GÜNTHER, AND A. HEINECKE, *Hardware-oriented implementation of cache oblivious matrix operations based on space-filling curves*, in Parallel Processing and Applied Mathematics, Springer, Berlin, 2008, pp. 628–638.
- [10] D. H. BAILEY, *Extra high speed matrix multiplication on the Cray-2*, SIAM J. Sci. Stat. Comput., 9 (1988), pp. 603–607.
- [11] D. H. BAILEY, K. LEE, AND H. D. SIMON, *Using Strassen's algorithm to accelerate the solution of linear systems*, J. Supercomput., 4 (1991), pp. 357–371.
- [12] G. BALLARD, A. R. BENSON, A. DRUINSKY, B. LIPSHITZ, AND O. SCHWARTZ, *Improving the numerical stability of fast matrix multiplication*, SIAM J. Matrix Anal. Appl., 37 (2016), pp. 1382–1418.
- [13] G. BALLARD, J. DEMMEL, O. HOLTZ, B. LIPSHITZ, AND O. SCHWARTZ, *Brief announcement: Strong scaling of matrix multiplication algorithms and memory-independent communication lower bounds*, in Proceedings of the Twenty-fourth Annual ACM Symposium on Parallelism in Algorithms and Architectures, ACM, New York, 2012, pp. 77–79.
- [14] G. BALLARD, J. DEMMEL, O. HOLTZ, B. LIPSHITZ, AND O. SCHWARTZ, *Communication-optimal parallel algorithm for Strassen's matrix multiplication*, in Proceedings of the Twenty-fourth Annual ACM Symposium on Parallelism in Algorithms and Architectures, ACM, New York, 2012, pp. 193–204.
- [15] G. BALLARD, J. DEMMEL, O. HOLTZ, AND O. SCHWARTZ, *Minimizing communication in numerical linear algebra*, SIAM J. Matrix Anal. Appl., 32 (2011), pp. 866–901.
- [16] G. BALLARD, J. DEMMEL, O. HOLTZ, AND O. SCHWARTZ, *Graph expansion and communication costs of fast matrix multiplication*, J. ACM, 59 (2013), pp. 1–23.
- [17] G. BENIAMINI, N. CHENG, O. HOLTZ, E. KARSTADT, AND O. SCHWARTZ, *Sparsifying the Operators of Fast Matrix Multiplication Algorithms*, preprint, arXiv:2008.03759, 2020.
- [18] G. BENIAMINI AND O. SCHWARTZ, *Faster matrix multiplication via sparse decomposition*, in The 31st ACM Symposium on Parallelism in Algorithms and Architectures, ACM, New York, 2019, pp. 11–22.
- [19] A. R. BENSON AND G. BALLARD, *A framework for practical parallel fast matrix multiplication*, ACM SIGPLAN Notices, 50 (2015), pp. 42–53.

- [20] J. BERNTSEN, *Communication efficient matrix multiplication on hypercubes*, Parallel Comput., 12 (1989), pp. 335–342.
- [21] G. BILARDI AND L. DE STEFANI, *The I/O complexity of Strassen’s matrix multiplication with recomputation*, in Workshop on Algorithms and Data Structures, Springer, Cham, Switzerland, 2017, pp. 181–192.
- [22] L. S. BLACKFORD, J. CHOI, A. CLEARY, E. D’AZEVEDO, J. DEMMEL, I. DHILLON, J. DONGARRA, S. HAMMARLING, G. HENRY, A. PETITET, K. STANLEY, D. WALKER, AND R. C. WHALEY, *ScaLAPACK Users’ Guide*, SIAM, Philadelphia, 1997.
- [23] B. BOYER, J.-G. DUMAS, C. PERNET, AND W. ZHOU, *Memory efficient scheduling of Strassen-Winograd’s matrix multiplication algorithm*, in Proceedings of the 2009 International Symposium on Symbolic and Algebraic Computation, ACM, New York, 2009, pp. 55–62.
- [24] R. P. BRENT, *Algorithms for Matrix Multiplication*, Technical report Stan-CS-70-157, Department of Computer Science, Stanford University, Stanford, CA, 1970.
- [25] L. E. CANNON, *A Cellular Computer to Implement the Kalman Filter Algorithm*, Ph.D. thesis, Montana State University, Bozeman, MT, 1969.
- [26] M. CENK AND M. A. HASAN, *On the arithmetic complexity of Strassen-like matrix multiplications*, J. Symbolic Comput., 80 (2017), pp. 484–501.
- [27] J. COHEN AND M. ROTH, *On the implementation of Strassen’s fast multiplication algorithm*, Acta Inform., 6 (1976), pp. 341–355.
- [28] D. COPPERSMITH AND S. WINOGRAD, *On the asymptotic complexity of matrix multiplication*, SIAM J. Comput., 11 (1982), pp. 472–492.
- [29] D. COPPERSMITH AND S. WINOGRAD, *Matrix multiplication via arithmetic progressions*, J. Symbolic Comput., 9 (1990), pp. 251–280.
- [30] P. D’ALBERTO, M. BODRATO, AND A. NICOLAU, *Exploiting parallelism in matrix-computation kernels for symmetric multiprocessor systems: Matrix-multiplication and matrix-addition algorithm optimizations by software pipelining and threads allocation*, ACM Trans. Math. Software, 38 (2011), pp. 1–30.
- [31] J. DEMMEL, I. DUMITRIU, AND O. HOLTZ, *Fast linear algebra is stable*, Numer. Math., 108 (2007), pp. 59–91.
- [32] J. DEMMEL, D. ELIAHU, A. FOX, S. KAMIL, B. LIPSHITZ, O. SCHWARTZ, AND O. SPILLINGER, *Communication-optimal parallel recursive rectangular matrix multiplication*, in 2013 IEEE 27th International Symposium on Parallel and Distributed Processing, IEEE, Piscataway, NJ, 2013, pp. 261–272.
- [33] F. DESPREZ AND F. SUTER, *Impact of mixed-parallelism on parallel implementations of the Strassen and Winograd matrix multiplication algorithms*, Concurrency Comput. Pract. Experience, 16 (2004), pp. 771–797.
- [34] J. J. DONGARRA, J. DU CROZ, S. HAMMARLING, AND I. S. DUFF, *A set of level 3 basic linear algebra subprograms*, ACM Trans. Math. Software, 16 (1990), pp. 1–17.
- [35] C. C. DOUGLAS, M. HEROUX, G. SLISHMAN, AND R. M. SMITH, *GEMMW: A portable level 3 BLAS Winograd variant of Strassen’s matrix-matrix multiply algorithm*, J. Comput. Phys., 110 (1994), pp. 1–10.
- [36] B. DUMITRESCU, *Improving and estimating the accuracy of Strassen’s algorithm*, Numer. Math., 79 (1998), pp. 485–499.
- [37] E. ELMROTH, F. GUSTAVSON, I. JONSSON, AND B. KÅGSTRÖM, *Recursive blocked algorithms and hybrid data structures for dense matrix library software*, SIAM Rev., 46 (2004), pp. 3–45.
- [38] A. FAWZI, M. BALOG, A. HUANG, T. HUBERT, B. ROMERA-PAREDES, M. BAREKATAIN, A. NOVIKOV, F. J. R. RUIZ, J. SCHRITTWIESER, G. SWIRSZCZ, D. SILVER, D. HASSABIS, AND P. KOHLI, *Discovering faster matrix multiplication algorithms with reinforcement learning*, Nature, 610 (2022), pp. 47–53.
- [39] M. FRIGO, C. E. LEISERSON, H. PROKOP, AND S. RAMACHANDRAN, *Cache-oblivious algorithms*, in 40th Annual Symposium on Foundations of Computer Science, IEEE Computer Society, Los Alamitos, CA, 1999, pp. 285–297.
- [40] P. GEPNER, V. GAMAYUNOV, AND D. L. FRASER, *Effective implementation of DGEMM on modern multicore CPU*, Procedia Comput. Sci., 9 (2012), pp. 126–135.
- [41] B. GRAYSON AND R. VAN DE GEIJN, *A high performance parallel Strassen implementation*, Parallel Process. Lett., 6 (1996), pp. 3–12.
- [42] T. HADAS AND O. SCHWARTZ, *Towards practical fast matrix multiplication based on trilinear aggregation*, in Proceedings of the 48th International Symposium on Symbolic and Algebraic Computation, ACM, New York, 2023.
- [43] N. J. HIGHAM, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, Philadelphia, 2002.

- [44] J. E. HOPCROFT AND L. R. KERR, *On minimizing the number of multiplications necessary for matrix multiplication*, SIAM J. Appl. Math., 20 (1971), pp. 30–36.
- [45] J. HUANG, L. RICE, D. A. MATTHEWS, AND R. A. VAN DE GEIJN, *Generating families of practical fast matrix multiplication algorithms*, in 2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS), IEEE, Piscataway, NJ, 2017, pp. 656–667.
- [46] J. HUANG, T. M. SMITH, G. M. HENRY, AND R. A. VAN DE GEIJN, *Strassen’s algorithm reloaded*, in SC’16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, IEEE, Piscataway, NJ, 2016, pp. 690–701.
- [47] J. HUANG, C. D. YU, AND R. A. V. D. GEIJN, *Strassen’s algorithm reloaded on GPUs*, ACM Trans. Math. Software, 46 (2020), pp. 1–22.
- [48] S. HUSS-LEDERMAN, E. M. JACOBSON, J. R. JOHNSON, A. TSAO, AND T. TURNBULL, *Implementation of Strassen’s algorithm for matrix multiplication*, in Supercomputing’96: Proceedings of the 1996 ACM/IEEE Conference on Supercomputing, IEEE, Piscataway, NJ, 1996, pp. 32–32.
- [49] S. HUSS-LEDERMAN, E. M. JACOBSON, J. R. JOHNSON, A. TSAO, AND T. TURNBULL, *Strassen’s algorithm for matrix multiplication: Modeling, analysis, and implementation*, in Proceedings of Supercomputing, IEEE, Piscataway, NJ, 1996, pp. 31–es.
- [50] D. IRONY, S. TOLEDO, AND A. TISKIN, *Communication lower bounds for distributed-memory matrix multiplication*, J. Parallel Distrib. Comput., 64 (2004), pp. 1017–1026.
- [51] R. W. JOHNSON AND A. M. MCLOUGHLIN, *Noncommutative bilinear algorithms for 3 x 3 matrix multiplication*, SIAM J. Comput., 15 (1986), pp. 595–603.
- [52] N. P. JOUPPI, D. H. YOON, M. ASHCRAFT, M. GOTTSCHO, T. B. JABLIN, G. KURIAN, J. LAUDON, S. LI, P. MA, X. MA, ET AL., *Ten lessons from three generations shaped Google’s TPuv4i*, in 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA), IEEE, Piscataway, NJ, 2021, pp. 1–14.
- [53] I. KAPORIN, *The aggregation and cancellation techniques as a practical tool for faster matrix multiplication*, Theoret. Comput. Sci., 315 (2004), pp. 469–510.
- [54] M. KARPPA AND P. KASKI, *Engineering Boolean Matrix Multiplication for Multiple-Accelerator Shared-Memory Architectures*, preprint, arXiv:1909.01554, 2019.
- [55] E. KARSTADT AND O. SCHWARTZ, *Matrix multiplication, a little faster*, J. ACM, 67 (2020), pp. 1–31.
- [56] A. KRECZMAR, *On memory requirements of Strassen’s algorithms*, in International Symposium on Mathematical Foundations of Computer Science, Springer, Berlin, 1976, pp. 404–407.
- [57] A. G. KRISHNAN AND D. GOSWAMI, *Multi-stage memory efficient Strassen’s matrix multiplication on GPU*, in 2021 IEEE 28th International Conference on High Performance Computing, Data, and Analytics (HiPC), IEEE, Piscataway, NJ, 2021, pp. 212–221.
- [58] B. KUMAR, C.-H. HUANG, P. SADAYAPPAN, AND R. W. JOHNSON, *A tensor product formulation of Strassen’s matrix multiplication algorithm with memory reduction*, Sci. Program., 4 (1995), pp. 275–289.
- [59] G. KWASNIEWSKI, M. KABIĆ, M. BESTA, J. VANDEVONDELE, AND R. SOLCÀ, *Red-blue pebbling revisited: Near optimal parallel matrix-matrix multiplication*, in Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC’19), Association for Computing Machinery, ACM, New York, 2019, 24.
- [60] J. LADERMAN, V. PAN, AND X.-H. SHA, *On practical algorithms for accelerated matrix multiplication*, Linear Algebra Appl., 162 (1992), pp. 557–588.
- [61] J. D. LADERMAN, *A noncommutative algorithm for multiplying 3x3 matrices using 23 multiplications*, Bull. Amer. Math. Soc., 82 (1976), pp. 126–128.
- [62] P.-W. LAI, H. ARAFAT, V. ELANGO, AND P. SADAYAPPAN, *Accelerating Strassen-Winograd’s matrix multiplication algorithm on GPUs*, in 20th Annual International Conference on High Performance Computing, IEEE, Piscataway, NJ, 2013, pp. 139–148.
- [63] F. LE GALL, *Powers of tensors and fast matrix multiplication*, in Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation, ACM, New York, 2014, pp. 296–303.
- [64] B. LIPSHITZ, G. BALLARD, J. DEMMEL, AND O. SCHWARTZ, *Communication-avoiding parallel Strassen: Implementation and performance*, in SC’12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis, IEEE, Piscataway, NJ, 2012, pp. 1–11.
- [65] Q. LUO AND J. B. DRAKE, *A scalable parallel Strassen’s matrix multiplication algorithm for distributed-memory computers*, in Proceedings of the 1995 ACM Symposium on Applied Computing, ACM, New York, 1995, pp. 221–226.

- [66] W. F. MCCOLL AND A. TISKIN, *Memory-efficient matrix multiplication in the BSP model*, *Algorithmica*, 24 (1999), pp. 287–297.
- [67] E. MEDINA AND E. DAGAN, *Habana labs purpose-built AI inference and training processor architectures: Scaling AI training systems using standard ethernet with Gaudi processor*, *IEEE Micro*, 40 (2020), pp. 17–24.
- [68] R. NISSIM AND O. SCHWARTZ, *Revisiting the I/O-complexity of fast matrix multiplication with recomputations*, in 2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS), IEEE, Piscataway, NJ, 2019, pp. 482–490.
- [69] V. Y. PAN, *Strassen’s algorithm is not optimal trilinear technique of aggregating, uniting and canceling for constructing fast algorithms for matrix operations*, in 19th Annual Symposium on Foundations of Computer Science, IEEE Computer Society, Long Beach, CA, 1978, pp. 166–176.
- [70] V. Y. PAN, *New fast algorithms for matrix operations*, *SIAM J. Comput.*, 9 (1980), pp. 321–342.
- [71] V. Y. PAN, *Trilinear aggregating with implicit canceling for a new acceleration of matrix multiplication*, *Comput. Math. Appl.*, 8 (1982), pp. 23–34.
- [72] R. L. PROBERT, *On the additive complexity of matrix multiplication*, *SIAM J. Comput.*, 5 (1976), pp. 187–203.
- [73] J. SCOTT, O. HOLTZ, AND O. SCHWARTZ, *Matrix multiplication I/O-complexity by path routing*, in Proceedings of the 27th ACM symposium on Parallelism in Algorithms and Architectures, ACM, New York, 2015, pp. 35–45.
- [74] A. V. SMIRNOV, *The bilinear complexity and practical algorithms for matrix multiplication*, *Comput. Math. Math. Phys.*, 53 (2013), pp. 1781–1795.
- [75] A. V. SMIRNOV, *Several Bilinear Algorithms for Matrix Multiplication*, Technical report, 2017, https://www.researchgate.net/publication/313064941_Several_bilinear_algorithms_for_matrix_multiplication.
- [76] E. SOLOMONIK, A. BHATELE, AND J. DEMMEL, *Improving communication performance in dense linear algebra via topology aware collectives*, in SC’11: Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis, IEEE, Piscataway, NJ, 2011, pp. 1–11.
- [77] E. SOLOMONIK AND J. DEMMEL, *Communication-optimal parallel 2.5D matrix multiplication and lu factorization algorithms*, in European Conference on Parallel Processing, Springer, Berlin, 2011, pp. 90–109.
- [78] A. J. STOTHERS, *On the Complexity of Matrix Multiplication*, Ph.D. thesis, University of Edinburgh, Edingburgh, UK, 2010.
- [79] V. STRASSEN, *Gaussian elimination is not optimal*, *Numer. Math.*, 13 (1969), pp. 354–356.
- [80] V. STRASSEN, *The asymptotic spectrum of tensors and the exponent of matrix multiplication*, in 27th Annual Symposium on Foundations of Computer Science, IEEE Computer Society, Los Angeles, CA, 1986, pp. 49–54.
- [81] Y. TANG, *Balanced partitioning of several cache-oblivious algorithms*, in Proceedings of the 32nd ACM Symposium on Parallelism in Algorithms and Architectures, ACM, New York, 2020, pp. 575–577.
- [82] M. THOTTETHODI, S. CHATTERJEE, AND A. R. LEBECK, *Tuning Strassen’s matrix multiplication for memory efficiency*, in SC’98: Proceedings of the 1998 ACM/IEEE Conference on Supercomputing, IEEE, Piscataway, NJ, 1998, pp. 36–36.
- [83] L. N. TREFETHEN AND D. BAU, *Numerical Linear Algebra*, SIAM, Philadelphia, 2022.
- [84] R. A. VAN DE GEIJN AND J. WATTS, *SUMMA: Scalable universal matrix multiplication algorithm*, *Concurrency Pract. Experience*, 9 (1997), pp. 255–274.
- [85] V. V. WILLIAMS, *Multiplying matrices faster than Coppersmith-Winograd*, in Proceedings of the Forty-fourth Annual ACM Symposium on Theory of Computing, ACM, New York, 2012, pp. 887–898.
- [86] S. WINOGRAD, *Private communication with R. Probert*, 1976.
- [87] D. S. WISE, *Ahnentafel indexing into Morton-ordered arrays, or matrix locality for free*, in European Conference on Parallel Processing, Springer, Berlin, 2000, pp. 774–783.